

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA
CORSO DI SOFTWARE ARCHITECTURE DESIGN
PROF.ESSA A.R. FASOLINO - A.A. 2025 - 26

Progetto Task R5

Modalità di Gioco Scalata

Gruppo D1: ingegneri dell'ascesa

Studenti:

Alessandro	Aversa	DE9000005	Aless.aversa@studenti.unina.it
Antonio	Di Giorgio	DE9000132	Antoni.digiorgio@studenti.unina.it
Alessandro	Cioffi	DE9000106	Alessand.cioffi@studenti.unina.it
Daniele	Liguori	DE9000072	danie.liguori@studenti.unina.it

URL progetto: <https://github.com/Testing-Game-SAD-2023/A13>

URL consegna: <https://github.com/MisterCioffi/A13>



Indice

1. DEFINIZIONE OBIETTIVI	3
1.1 DESCRIZIONE SINTETICA DEL TASK ASSEGNATO	3
1.2 SPECIFICHE INFORMALI	3
2. ANALISI E SPECIFICA DEI REQUISITI	4
2.1 GLOSSARIO DEI TERMINI	4
2.2 CLASSIFICAZIONE DEI REQUISITI.....	4
2.2.1 Requisiti funzionali.....	4
2.2.2 Requisiti sui dati	5
2.2.3 Vincoli / Altri requisiti	5
2.3 MODELLAZIONE DEI CASI D'USO	6
2.3.1 Attori e casi d'uso	6
2.3.2 Diagramma dei casi d'uso	2
2.3.3 Scenari	2
2.4 MODELLAZIONE DEI DATI	5
2.4.1 Progettazione concettuale	5
2.5 DIAGRAMMA DELLE CLASSI.....	5
2.5.1 Diagramma di espansione del modulo T1.....	5
2.6 IMPLICAZIONI ARCHITETTURALI.....	6
2.6.1 Modulo T1	6
2.6.2 Modulo T4	7
2.6.3 Modulo T56.....	7
2.7 COOPERAZIONE INTER-TASK.....	8
2.8 APPROCCIO DI SVILUPPO	9
3. PROGETTAZIONE.....	10
3.1 PROGETTAZIONE DELLA BASE DI DATI.....	10
3.1.1 Progettazione logica (<i>modulo T4</i>).....	10
3.2 DIAGRAMMA DELLE CLASSI (<i>MODULO T56</i>).....	13
3.2.1 Diagramma di " <i>GameLogic</i> "	13
3.2.2 Diagramma di " <i>GameManager&Controller</i> "	16
3.3 DIAGRAMMA DI SEQUENZA.....	20
3.3.1 Diagramma di " <i>HandleLevelWon</i> "	20
3.3.2 Diagramma di " <i>HandleLevelLost</i> "	21
3.3.3 Diagramma di " <i>SelezionaScalata</i> "	21
3.4 GESTIONE DELLE INTERFACCE: <i>ESTENSIONI E MODIFICHE</i>	22
3.4.1 Lista Scalate	22
3.4.2 Recupero Dettagli Livello Scalata.....	23
3.4.3 Incremento del <i>currentlevel</i> di una partita.....	24
3.4.4 Incremento Tentativo Round [<i>Flusso: T56 (GameEngine) -> T4 (GameRepo)</i>]	25



4. IMPLEMENTAZIONE.....	26
4.1 PACKAGE “GAME/GAMEDTO”	26
4.1.1 Package “RunGameDTO”	27
4.1.2 Package “CreateSessionDTO”	28
4.1.3 Package “EndGameDTO”	29
4.1.4 Package “StartGameDTO”	30
4.2 PACKAGE “GAME/GAMEFACTORY”	31
4.2.1 Package “gameFactory”	31
4.2.2 Package “params”	32
4.3 PACKAGE “MODEL/DTO”	33
5. SVILUPPI FUTURI	34
6. TESTING.....	36
6.1 TEST FUNZIONALE	36

1. Definizione obiettivi

1.1 Descrizione sintetica del task assegnato

Il *Task R5* propone di creare la *modalità Scalata*, in cui il giocatore affronta una serie di livelli consecutivi gestiti dall'amministratore, consistenti nello sviluppo di test *JUnit*.

Al completamento di ogni livello vengono assegnati punti, e il superamento del livello finale garantisce bonus e achievement.

La modalità, inoltre, permette di riprovare i livelli falliti assegnando penalità o terminare in qualsiasi momento la scalata.

1.2 Specifiche informali

La modalità di **Gioco Scalata** prevede di superare una sequenza di livelli consecutivi. Il giocatore dopo aver selezionato tale modalità avrà l'opportunità di selezionare una tra le varie scalate offerte dal sistema.

Ogni partita della modalità Scalata è composta da un numero prefissato di livelli, uguale per tutti i giocatori.

Ogni livello può essere composto da uno o più round in base alle abilità del giocatore. In ciascun round il giocatore deve scrivere test *JUnit* su una diversa classe Java fornita dal sistema.

L'obiettivo del giocatore è ottenere, in ciascun round, una copertura di codice uguale o superiore a quella richiesta nel tempo previsto, così da poter avanzare nella scalata.

La modalità Scalata utilizza la stessa interfaccia grafica di gioco della modalità *Classica*, ma differisce nel flusso di gioco: una volta raggiunta o superata la copertura richiesta nel tempo previsto, il giocatore, invece di terminare la partita come nella modalità *Classica*, può decidere di avviare un nuovo livello, caratterizzato da una nuova coppia classe/avversario.

La difficoltà dei round è incrementale e prestabilita dall'amministratore del sistema, il quale definisce anche la composizione di ciascun livello: la classe assegnata e il relativo avversario.

Qualora il giocatore decidesse di mettere in pausa la partita, potrà farlo in qualsiasi momento. Successivamente, potrà riprendere la scalata esattamente dal punto in cui l'aveva interrotta. Al termine di ogni livello completato vengono assegnati dei punti.

In caso di terminazione della partita senza completare la scalata, il giocatore mantiene i punti relativi ai livelli completati, ma non riceve ulteriori ricompense.

Completando con successo tutti i livelli previsti, il giocatore ottiene un punteggio bonus e una ricompensa speciale (Achievement).

Nel profilo del giocatore deve comparire l'Achievement relativo alla partita vinta e i punti guadagnati.

Il round è considerato fallito quando il tempo è scaduto oppure quando il giocatore decide di terminare la partita senza aver raggiunto la copertura minima.

Quando il round è fallito, la scalata non viene interrotta: in qualsiasi momento il giocatore potrà decidere di riprovare il livello fallito avviando un nuovo round. Tuttavia, in tal caso, il giocatore riceverà una penalità alla fine della scalata.

Il sistema, inoltre, deve consentire al giocatore di resettare la partita e di ricominciare la scalata dal primo livello. La gestione delle varie scalate è affidata all'amministratore, il quale avrà il compito di configurare e sovrintendere le varie scalate.

2. Analisi e specifica dei requisiti

2.1 Glossario dei termini

Termine	Descrizione	Sinonimi
Scalata	Identifica un insieme di livelli	Partita, gioco
Livello	Identifica un insieme di round caratterizzati dalla stessa coppia <i>Classe/Avversario</i>	
Round	Identifica una singola sfida contro un avversario su una determinata classe	Tentativo
Penalità	Rimozione specifica di punti	
Amministratore	Ruolo di gestione del sistema	

2.2 Classificazione dei requisiti

2.2.1 Requisiti funzionali

ID	Requisito
RF01	Il sistema deve consentire al giocatore di accedere alla modalità di Gioco Scalata
RF02	Il sistema deve consentire al giocatore di visualizzare le scalate presenti nel sistema
RF03	Il sistema deve consentire al giocatore di selezionare quale scalata giocare
RF04	Il sistema deve gestire per ogni scalata, una sequenza di livelli consecutivi, ognuno dei quali presenta una coppia diversa <i>classeJava/avversario</i> su cui scrivere test <i>JUnit</i>
RF05	Il sistema deve poter offrire la possibilità di calcolare la copertura di codice ottenuta dal giocatore
RF06	Il sistema deve poter verificare se la copertura ottenuta è uguale o superiore a quella richiesta
RF07	Il sistema deve consentire al giocatore di avviare un nuovo round del nuovo livello una volta raggiunta o superata la copertura richiesta nel tempo richiesto
RF08	Il sistema deve consentire al giocatore di mettere in pausa la partita in qualsiasi momento e di riprenderla successivamente dal punto in cui era stata interrotta
RF09	Il sistema deve consentire che in caso di fallimento di un round il giocatore riceva una penalità alla fine della scalata per quel livello specifico
RF10	Il sistema deve consentire in caso di fallimento di un round, al giocatore di riprendere in qualsiasi momento la partita giocando un nuovo round del livello fallito

RF11	Il sistema deve salvare automaticamente lo stato della partita al termine di ogni round
RF12	Il sistema deve assegnare punti al termine di ogni livello superato
RF13	Il sistema deve attribuire un punteggio bonus e una ricompensa speciale (Achievement) al completamento di tutti i livelli previsti
RF14	Il sistema deve permettere all'amministratore di configurare la difficoltà dei livelli, le classi assegnate e la composizione complessiva delle scalate

2.2.2 Requisiti sui dati

ID	Requisito
RD01	Ogni livello in cui almeno un round è fallito può assegnare al più una penalità al giocatore
RD02	Il sistema deve memorizzare per ogni livello la classe Java assegnata, l'avversario e il tempo massimo previsto per ogni round del livello
RD03	Il sistema deve salvare lo stato della partita (round attuale con le relative informazioni: punteggio, tempo rimanente, copertura ottenuta e i casi di test scritti dall'utente) quando il giocatore decide di mettere in pausa
RD04	Il sistema deve memorizzare inoltre per ogni livello il numero di round sostenuti dal giocatore e i suoi relativi risultati
RD05	Il sistema deve memorizzare i punteggi complessivi e le ricompense acquisite dal giocatore al termine della partita tenendo conto di eventuali penalità

2.2.3 Vincoli / Altri requisiti

ID	Requisito
Vo1	Il round è definito fallito quando il tempo termina, oppure il giocatore decide di concludere il round ma la soglia di copertura del giocatore è minore di quella dell'avversario
Vo2	Un livello è considerato fallito se tutti i round svolti dal giocatore per quel determinato livello sono falliti
Vo3	Le eventuali penalità vengono assegnate al giocatore alla fine della scalata, cioè solo in caso di superamento di tutti i livelli
Vo4	Il numero di livelli di una singola Scalata deve essere prefissato e uguale per tutti i giocatori
Vo5	La difficoltà dei livelli deve aumentare in modo progressivo secondo la configurazione stabilita dall'amministratore
Vo6	Solo nel caso in cui un round è considerato non fallito il giocatore può accedere al livello successivo

V07	In caso di fallimento di un livello, il giocatore inizierà un nuovo round relativo allo stesso livello quando il giocatore riterrà più opportuno
V08	Il sistema deve garantire la persistenza dei dati di gioco anche in caso di uscita o interruzione improvvisa dell'applicazione
V09	Il sistema deve garantire che una scalata può essere modificata solo se nessun giocatore in quel momento ha una partita in corso
V10	Il sistema deve garantire che ogni giocatore può avere “in corso” solo una scalata alla volta

2.3 Modellazione dei casi d'uso

2.3.1 Attori e casi d'uso

Attori Primari:

- Giocatore
- Amministratore
- Tempo

Casi d'uso:

- **UCo1:** avvia modalità scalata
- **UCo2:** Gestione Progressione Livelli
- **UCo3:** Gestisci scalata

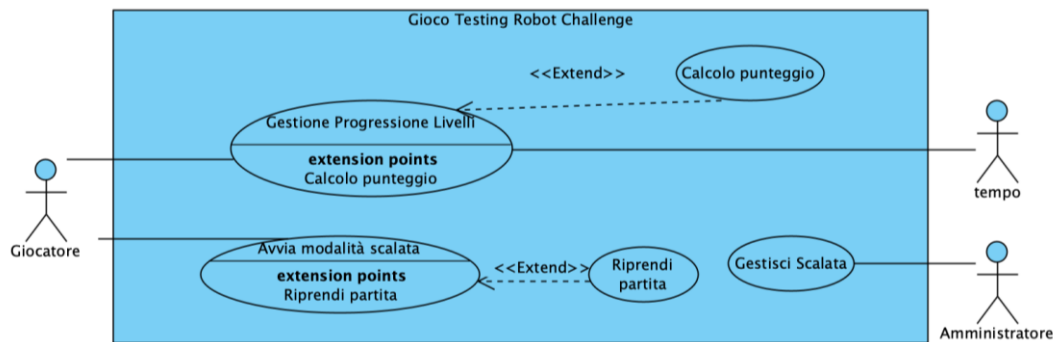
Casi d'uso di inclusione:

Casi d'uso di estensione:

- **UCo4:** Riprendi partita
- **UCo5:** Calcolo punteggio

Caso d'uso	Attori Primari	Attori Secondari	Incl. / Ext.
Avvia modalità scalata	Giocatore	-	Esteso da: <i>Riprendi partita</i>
Riprendi partita	-	-	Estende: <i>Avvia modalità scalata</i>
Gestione Progressione Livelli	Giocatore e Tempo	-	Esteso da: <i>Calcolo punteggio</i>
Calcolo punteggio	-	-	Estende: <i>Gestione Progressione Livelli</i>
Gestisci Scalata	Amministratore	-	-

2.3.2 Diagramma dei casi d'uso



2.3.3 Scenari

Caso d'uso:	Avvia modalità scalata
Attore primario	Giocatore
Attore secondario	-
Descrizione	Il caso d'uso gestisce l'ingresso dell'utente nella sezione "Scalata". Il sistema offre al giocatore la possibilità di riprendere l'ultima partita sospesa (se presente) o iniziarne una nuova.
Pre-Condizioni	- L'utente è autenticato - Sia presente almeno una scalata nel sistema
Sequenza di eventi principale	- Il caso d'uso inizia quando il giocatore seleziona la voce "Modalità Scalata" dal menu principale. - Il sistema verifica la presenza di una partita in sospeso IF non esiste una partita in corso (<i>else sequenza alternativa</i>) - Il sistema recupera l'elenco delle Scalate disponibili. - Il sistema mostra la pagina di selezione, presentando per ogni Scalata le informazioni rilevanti. - Il giocatore seleziona la Scalata che desidera intraprendere. - Il sistema crea una nuova partita associata al giocatore. - Il sistema predispone l'ambiente di gioco caricando i dati del Livello 1. - Il sistema mostra la pagina principale di gioco pronta per il primo round.
Post-Condizioni	Il sistema mostra al giocatore la pagina coerente con la decisione presa dallo stesso.
Casi d'uso correlati	<i>Riprendi partita</i>
Sequenza di eventi alternativi	Al punto 3, se il sistema rileva una partita in corso: IF il giocatore decide di abbandonare la partita: - Il sistema elimina la partita in corso - Il sistema riporta il giocatore alla schermata principale IF il giocatore decide di riprendere la partita: Punto di estensione: "Riprendi Partita".

Caso d'uso:	Riprendi partita
Attore primario	-
Attore secondario	-
Descrizione	Il sistema deve consentire al giocatore di poter riprendere la scalata lasciata in sospeso.
Pre-Condizioni	Deve esistere una scalata in corso associata al giocatore
Sequenza di eventi principale	1. Il sistema verifica lo stato della partita memorizzata. 2. IF il livello corrente risulta completato: 2.1. Il sistema predisporre i dati per il livello successivo. 2.2. Il sistema avvia un nuovo round relativo al livello raggiunto. 2.3. Il sistema mostra la schermata di gioco relativa al round avviato.
Post-Condizioni	• Il sistema ha caricato l'editor di gioco relativo ad un determinato round. • Per quel determinato giocatore esiste un round in corso.
Casi d'uso correlati	-
Sequenza di eventi alternativi	Al punto 2, se il livello non è ancora concluso: 2.1. Il sistema recupera i dati esatti del livello interrotto (<i>classe di test scritta dall'utente, classe under test, opponent, tempo rimanente al momento dell'interruzione</i>).

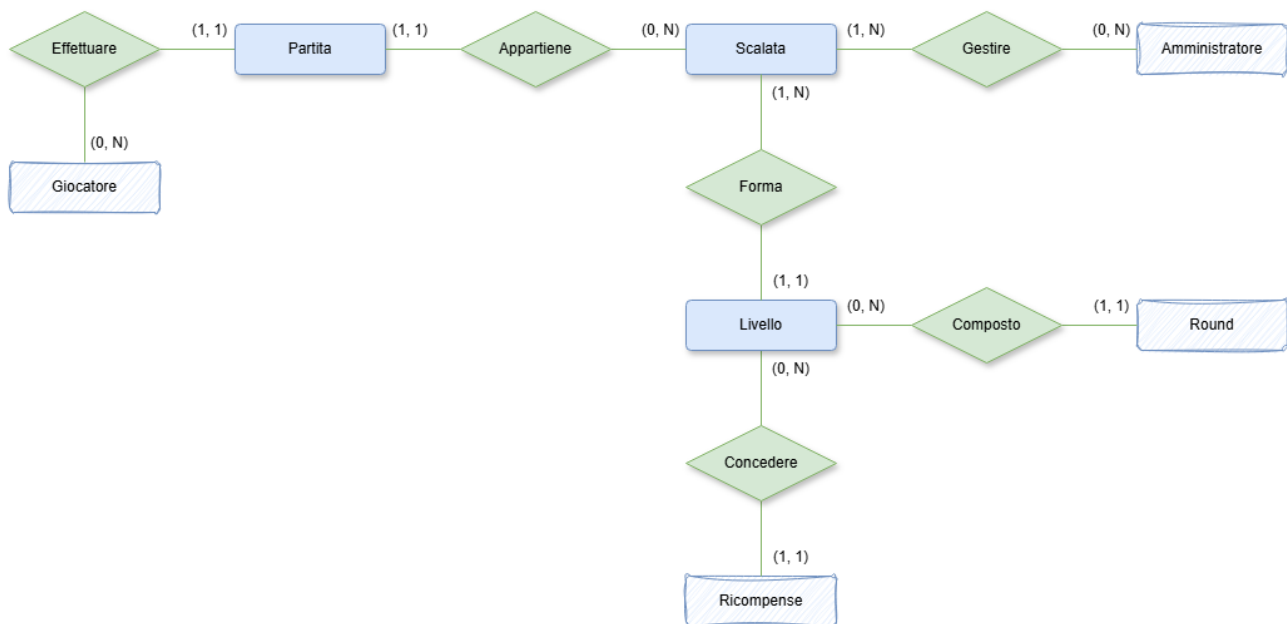
Caso d'uso:	Gestione progressione livelli
Attore primario	Giocatore e Tempo
Attore secondario	-
Descrizione	Il sistema valuta l'esito di un <u>round</u> di gioco all'interno di una Scalata, determinando se il giocatore può avanzare al livello successivo, deve ripetere il livello corrente, o se la scalata è conclusa.
Pre-Condizioni	- Il giocatore ha una "Scalata" attiva. - Il giocatore ha un round non ancora concluso.
Sequenza di eventi principale	- Il caso d'uso inizia quando il giocatore decide di terminare definitivamente il round, oppure il tempo scade. - Il sistema confronta le metriche ottenute dal giocatore con i criteri di vittoria del livello (<u>Se i criteri non sono soddisfatti vedi alternativa 3a</u>). - Il sistema verifica se il livello appena superato è l'ultimo della scalata: 3.1. IF esiste un livello successivo (<i>vittoria intermedia</i>): 3.1.1. Il sistema memorizza i risultati ottenuti dal giocatore. 3.1.2. Il Sistema mostra al Giocatore un messaggio di "<i>Livello Superato</i>". 3.1.3. Il sistema aggiorna lo stato della scalata e predispone i parametri per il livello successivo. 3.1.4. Il sistema permette al giocatore di accedere al round del livello successivo. 3.2. IF non esiste un livello successivo (<i>vittoria finale</i>): 3.2.1. Punto di estensione: "<i>Calcolo Punteggio</i>" per determinare il punteggio finale della Scalata. 3.2.2. Il Sistema registra la Scalata come completata.
Post-Condizioni	Una scalata è memorizzata con lo stato " <i>in corso</i> " oppure " <i>terminata</i> " per quel determinato giocatore
Casi d'uso correlati	<i>Calcolo punteggio</i>
Sequenza di eventi alternativi	- Il sistema incrementa il numero di tentativi di quel livello. - Il sistema notifica il fallimento al giocatore. - Il sistema predispone un nuovo Round dello stesso livello.

N.B.

Per *pagina principale di gioco* si intende quella dove il giocatore è chiamato a scrivere test *JUnit* (Editor).

2.4 Modellazione dei dati

2.4.1 Progettazione concettuale

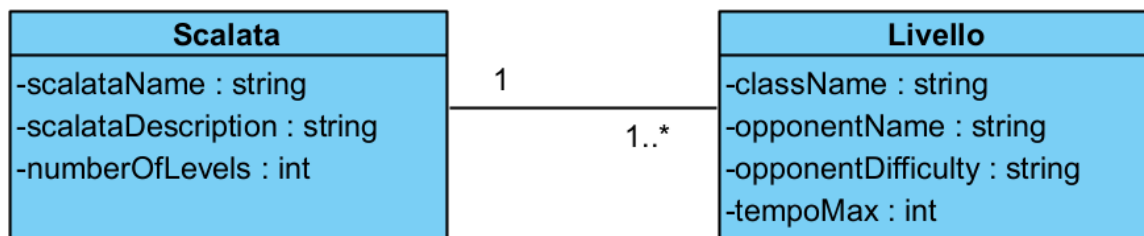


2.5 Diagramma delle classi

2.5.1 Diagramma di espansione del modulo T1

Di seguito si presenta il modello concettuale relativo alle entità **Scalata** e **Livello** all'interno del contesto del modulo **T1**, comprensivo dei rispettivi attributi e delle relazioni di interesse.

L'attività di implementazione, integrazione e ottimizzazione di tali entità con i dati già presenti nel modulo **T1** non rientra nel perimetro della presente analisi e sarà pertanto demandata al gruppo designato a tale scopo (*Task R1*).



Si ricorda che il modulo **T1** è responsabile della gestione dell'interfaccia utente e dei servizi dedicati agli amministratori, con particolare riferimento alla gestione delle classi, dei robot e delle relative configurazioni.

2.6 Implicazioni Architetture

La presente sezione analizza gli impatti che l'introduzione della *modalità Scalata* comporta sull'architettura esistente del sistema.

In particolare, vengono individuati i moduli coinvolti, le modifiche necessarie e le potenziali ripercussioni strutturali e funzionali sul *framework* attuale.

L'obiettivo è delineare in modo chiaro e completo il perimetro delle evoluzioni richieste, fornendo una base solida per le successive fasi di progettazione e implementazione.

I moduli coinvolti nelle modifiche sono:

- **Modulo T1:** *gestione amministrativa*
- **Modulo T4:** *persistenza dati di gioco*
- **Modulo T56:** *logica di gioco e implementazione UI*

2.6.1 Modulo T1

Il modulo **T1** (*Gestito dal task R1*), responsabile della gestione amministrativa, dovrà essere esteso per includere le funzionalità di definizione e configurazione della *modalità Scalata*. Tali estensioni riguarderanno principalmente la gestione dei *livelli*, delle classi associate e degli *avversari*, in coordinamento con le attività previste dal *Task R4*.

Modifiche Necessarie

- **Introduzione del modello dati della Scalata:** definizione delle entità, degli attributi e delle strutture dati necessarie a rappresentare *Scalate* e *Livelli*, garantendo coerenza con gli standard già adottati dal modulo *T1*
- **Estensione delle funzionalità UI e API:** adeguamento dell'interfaccia utente amministrativa e dei servizi esposti affinché consentano la creazione, configurazione e consultazione delle *Scalate*

Impatto Complessivo

- **Coerenza dei dati verso T5:** il modulo *T1* dovrà esporre dati completi, strutturati e consistenti affinché il modulo *T5* possa eseguire correttamente la logica di progressione dei *livelli* della *Scalata*
- **Integrazione non invasiva nell'architettura esistente:** l'introduzione della *modalità Scalata* dovrà essere incorporata senza interferire con le funzionalità già operative per la gestione di classi, robot e configurazioni amministrative
- **Incremento della complessità gestionale:** l'ampliamento delle responsabilità del modulo *T1* comporta un aumento della complessità funzionale e richiede una revisione delle interfacce e dei flussi di configurazione per assicurare uniformità e manutenibilità

2.6.2 Modulo T4

Il modulo **T4**, responsabile della persistenza delle *partite* e dei *round*, richiede una revisione mirata per supportare la *modalità Scalata* e la correlata logica di progressione *multi-round*. In particolare, il *repository* delle *partite* dovrà essere adattato affinché sia possibile registrare in modo incrementale lo stato del *giocatore* lungo la sequenza di *livelli* prevista dalla *Scalata*.

Modifiche Necessarie

- **Verifica e adeguamento della struttura dati esistente:** analisi della compatibilità del modello di persistenza attuale con la logica sequenziale della *Scalata*, con eventuali interventi strutturali
- **Aggiornamento delle operazioni di salvataggio:** revisione dei metodi di persistenza per garantire la registrazione coerente e progressiva dei *round* e dei *livelli*, assicurando l'allineamento con i requisiti gestionali del modulo T5
- **Eventuali modifiche alla struttura dati esistente:** introduzione di eventuali attributi o relazioni aggiuntive necessarie a supportare in modo completo le funzionalità previste dal modello concettuale della *Scalata*

Impatto Complessivo

- **Tracciabilità completa dello stato di avanzamento:** il modulo T4 dovrà garantire la registrazione accurata dell'evoluzione del giocatore attraverso *livelli* e *round*, supportando correttamente la logica di progressione definita da T5
- **Coerenza intermodulare:** tutte le modifiche dovranno preservare la coerenza con i moduli esistenti e con le *API* attualmente esposte, evitando effetti collaterali sulla persistenza delle modalità di gioco già operative
- **Interventi a basso impatto architettuale:** l'adeguamento richiesto è limitato e non comporta una revisione profonda della struttura del modulo T4, garantendo continuità con l'architettura di persistenza esistente

2.6.3 Modulo T56

Il modulo **T56**, che costituisce il nucleo della logica di gioco e dell'interfaccia utente dedicata al *giocatore*, rappresenta il componente maggiormente coinvolto dall'introduzione della *modalità Scalata*. Tale modulo dovrà essere esteso per orchestrare la progressione *multi-livello*, gestire le transizioni tra i *round* e garantire una presentazione coerente e responsiva delle informazioni all'interno dell'esperienza di gioco.

Modifiche Necessarie

- **Introduzione della logica di gestione della Scalata:** integrazione di un flusso strutturato che permetta la sequenza ordinata di più *livelli* consecutivi, con verifica delle condizioni di completamento e avanzamento
- **Estensione del Game Manager:** adeguamento dei componenti responsabili dell'orchestrazione del *gameplay* affinché coordinino correttamente la successione dei *round*, le transizioni tra *livelli* e l'interazione con i dati provenienti dai moduli T1 e T4
- **Aggiornamento dell'interfaccia utente del giocatore:** introduzione di nuovi elementi *UI* finalizzati a rappresentare in modo chiaro lo stato di avanzamento, il *livello* corrente, i risultati intermedi e il riepilogo complessivo della *Scalata*

Impatto Complessivo

- **Adattamento del flusso di gioco alla progressione *multi-round*:** l'intero ciclo di esecuzione della *partita* dovrà essere riallineato affinché supporti nativamente la logica di avanzamento tra *livelli*, preservando la coerenza delle interazioni esistenti
- **Integrazione strutturata con i moduli *T1* e *T4*:** il modulo *T56* dovrà recepire in maniera robusta i dati di configurazione forniti da *T1* e interagire correttamente con la persistenza gestita da *T4*, garantendo un flusso informativo stabile e sincronizzato
- **Consolidamento dell'esperienza di gioco:** l'introduzione della *modalità Scalata* richiederà un'evoluzione significativa dell'esperienza utente, assicurando continuità visiva e funzionale, e mantenendo elevati livelli di usabilità a fronte dell'aumento della complessità del *gameplay*

2.7 Cooperazione Inter-Task

Il nostro gruppo è responsabile della **progettazione e sviluppo della *modalità Scalata***, comprendente sia la definizione della logica di gioco che l'implementazione dei meccanismi necessari per orchestrare la progressione dei *livelli*, la gestione dei *round* e le interazioni del *giocatore* con la nuova modalità. In questo contesto, ci occupiamo della logica di gioco della *Scalata*, senza intervenire direttamente sulle modifiche del modulo ***T1*** o sulle funzionalità di gestione amministrativa, che sono affidate rispettivamente ai task ***R1*** e ***R4***.

- **Task *R1* – Implementazione tecnica delle modifiche *T1*:** il gruppo *R1* traduce in implementazioni operative le specifiche concettuali fornite dal nostro gruppo, occupandosi di tutte le modifiche necessarie al modulo *T1* per integrare la *modalità Scalata*. Ciò include la gestione dei dati relativi ai *livelli*, alle *classi* e agli *avversari*, garantendo coerenza con la logica di gioco
- **Task *R4* – Gestione amministrativa della *Scalata*:** il gruppo *R4* è responsabile dello sviluppo delle funzionalità amministrative memorizzate nel modulo *T1*, permettendo agli *amministratori* di configurare e controllare la *modalità Scalata*. L'implementazione delle interfacce e dei servizi dedicati rientra pienamente nelle loro competenze, seguendo le specifiche da noi definite

La collaborazione tra i gruppi si è basata su un **costante coordinamento e confronto**:

- Sono stati definiti in anticipo i confini di responsabilità e le interfacce comuni, così da permettere uno **sviluppo parallelo** senza conflitti tra i task
- Il nostro gruppo ha mantenuto il ruolo di **progettazione e implementazione della logica di gioco**, mentre *R1* e *R4* hanno gestito le modifiche a *T1* e le funzionalità amministrative
- Periodici incontri di allineamento hanno assicurato **coerenza funzionale e architetturale**, permettendo di integrare correttamente le componenti sviluppate dai diversi gruppi

2.8 Approccio di Sviluppo

Il progetto è stato condotto seguendo un **approccio iterativo basato su metodologie Agili**, con l'obiettivo di garantire flessibilità, rapidità di adattamento e continuo miglioramento del prodotto.

L'iterazione costante ha permesso al gruppo di sviluppare incrementi funzionali della *modalità Scalata*, testando progressivamente le componenti e integrandole con i moduli esistenti.

La gestione del progetto è stata organizzata secondo i principi fondamentali dell'*Agile*:

- **Pianificazione incrementale:** le attività sono state suddivise in *sprint*, ciascuno con obiettivi chiari e verificabili, permettendo un monitoraggio costante dello stato di avanzamento
- **Integrazione continua e feedback frequente:** il codice sviluppato è stato periodicamente integrato e testato, consentendo di identificare e correggere tempestivamente eventuali incongruenze o anomalie
- **Collaborazione e comunicazione costante:** il gruppo ha mantenuto un confronto regolare con i task *R1* e *R4*, garantendo il coordinamento necessario per uno sviluppo parallelo e coerente con l'architettura complessiva del sistema
- **Adattamento alle esigenze emergenti:** il modello iterativo ha consentito di rivedere e ottimizzare le soluzioni implementative sulla base dei risultati ottenuti e delle nuove informazioni emerse durante lo sviluppo

Inoltre, il nostro gruppo, composto da *4 membri*, ha adottato il ***pair programming*** durante lo sviluppo del codice. Ciò ha permesso di lavorare a coppie sulla stessa funzionalità, favorendo la condivisione delle conoscenze, il controllo reciproco del codice e un incremento della qualità complessiva del prodotto.

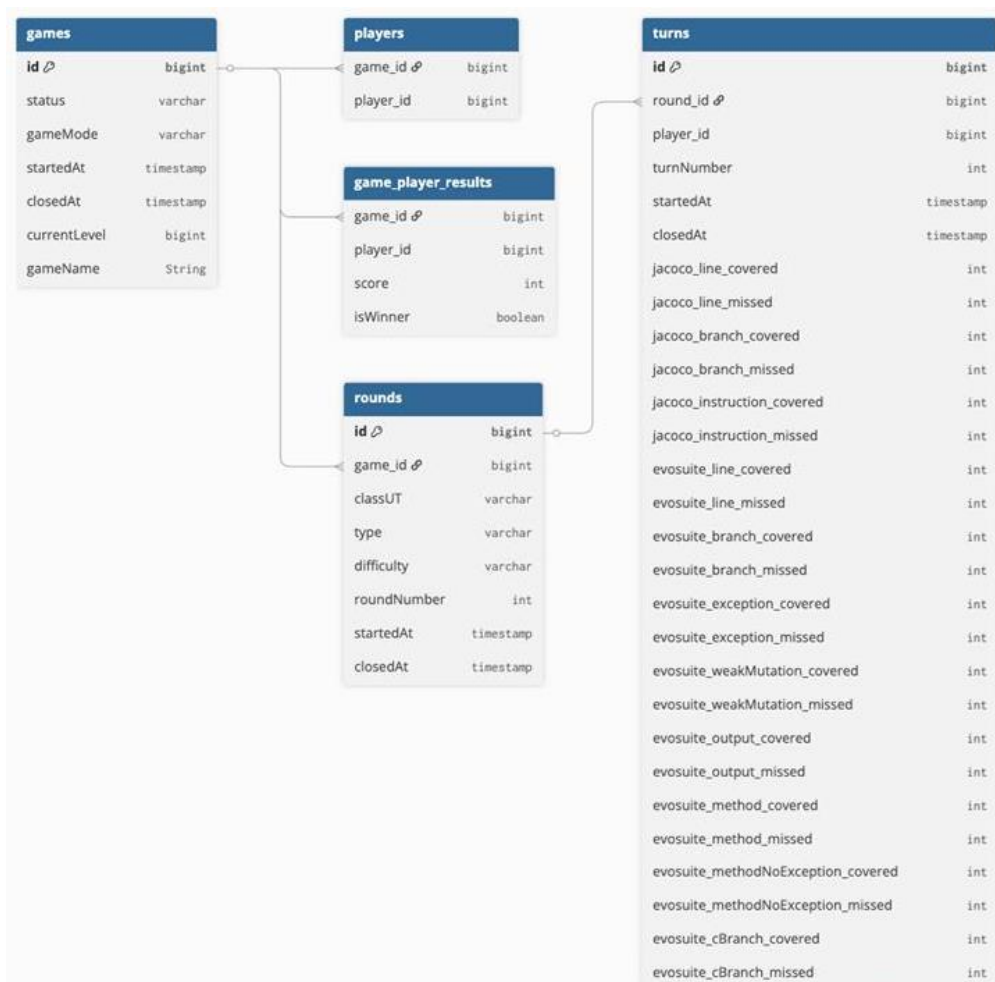
Per l'adozione del *pair programming*, il nostro gruppo si è ispirato principalmente alle pratiche di ***Extreme Programming (XP)***. Pur seguendo un approccio *Agile iterativo*, l'*XP* ci ha fornito indicazioni concrete su come organizzare le coppie, alternare i ruoli di ***Driver*** e ***Navigator*** e mantenere un controllo continuo sulla correttezza e coerenza del codice sviluppato.

In sintesi, l'approccio *Agile* adottato ha favorito una gestione dinamica e controllata del progetto, assicurando elevati livelli di qualità, coerenza architetturale e progressiva integrazione della nuova *modalità Scalata* all'interno del sistema esistente.

3. Progettazione

3.1 Progettazione della base di dati

3.1.1 Progettazione logica (modulo T4)



Ragioni tecniche alla base dell'evoluzione della struttura dati

L'introduzione della nuova *modalità Scalata* ha reso necessario estendere il modello dati del *container di T4* per consentire la persistenza riguardante la progressione del *giocatore* attraverso una sequenza strutturata di *livelli*.

Tale esigenza non era presente nelle modalità preesistenti (*Allenamento* e *Partita Singola*), le quali non prevedevano alcun avanzamento progressivo.

Per garantire la massima *retro-compatibilità*, l'intervento è stato progettato secondo un criterio conservativo, preservando l'architettura generale e limitando le modifiche ai soli elementi indispensabili all'introduzione della nuova dinamica di gioco.

Attenzione: ciascun *giocatore* può partecipare ad una sola *scalata* per volta; non è consentito avviare più *partite* simultanee per lo stesso *giocatore*.

Adattamento dell'architettura gerarchica Game → Round → Turn

La struttura gerarchica originaria è stata mantenuta, ma reinterpretata in modo da modellare correttamente i concetti fondamentali della *modalità Scalata*.

➤ Entità **Games** – Contenitore logico della *scalata*

Come nelle modalità esistenti, per ciascuna sessione viene creata una singola istanza dell'entità *Games*, che funge da contenitore logico dell'intera esperienza di gioco.

Per supportare la progressione *multi-livello* sono stati introdotti due attributi:

- **gameName (utilizzo futuro)**: campo identificativo necessario per gestire una partita in *modalità scalata* "in corso" in assenza di *sessione*. L'utilizzo futuro e le possibili estensioni di questo attributo sono approfonditi nel *capitolo 5*
- **currentLevel**: contatore che registra il *livello* attualmente raggiunto dal *giocatore*. Nelle *modalità legacy* tale valore è implicitamente pari a 1, poiché non è previsto alcun avanzamento

➤ Entità **Rounds** – Rappresentazione dei *livelli*

Non è stata introdotta una nuova entità *level*, poiché il concetto di *livello* è stato invece implementato attraverso l'entità *Rounds*, già presente nel modello.

In questo contesto un attributo già presente assume un ruolo fondamentale:

- **roundNumber**: esso identifica il *numero di tentativi* svolti per il *livello* corrente

Precisazioni:

- Il raggiungimento di ogni *livello* comporta la creazione di un nuovo *round*
- Ogni *tentativo* fallito di un *livello* non genera una nuova istanza di *Round*, ma **aggiorna** il *Round* relativo al *livello*, incrementando il valore del campo *roundNumber*

Questa scelta consente di integrare la logica della *scalata* senza alterare la struttura relazionale né introdurre nuove tabelle.

➤ Entità **Turn** – Struttura Invariata

L'entità **Turn** rappresenta l'unità atomica dell'interazione di gioco all'interno di un *Round*. Essa costituisce l'elemento fondamentale per tracciare lo svolgimento del *round* e valutare le prestazioni del giocatore.

L'entità **Turn** non ha richiesto alcuna modifica.

Il modello attuale risulta già pienamente coerente con le esigenze della *modalità Scalata*, e la gestione dei *turni* può essere mantenuta integralmente invariata rispetto alle modalità preesistenti.



Stati di Gioco e rispettive transizioni

Per supportare il funzionamento della modalità *Scalata*, il campo **status** dell'entità *Games* è stato aggiornato modificando il senso dei vari stati che descrivono con precisione le eventuali situazioni che possono presentarsi.

➤ **STARTED** — Stato di Inizializzazione

Stato assegnato quando un *livello* è stato caricato, ma al quale non corrisponde nessuno *turno*.

Tale stato viene impostato:

- All'avvio della *scalata* (*Livello 1*)
- Al completamento di ciascun *livello* e all'ingresso in quello successivo
 - **Esempio:** passaggio dal *Livello 1* al *Livello 2*

➤ **IN_PROGRESS** — Stato di *livello in corso*

Indica che il *giocatore* ha iniziato a interagire con il *livello* corrente.

Tale stato viene impostato quando:

- Il *giocatore* effettua il primo turno del *livello* corrente
 - Tale evento comporta la transizione dello *status* da *STARTED* a *IN_PROGRESS*

➤ **SURRENDERED** — Stato conclusivo per *Abbandono (resa)*

Indica la conclusione della *partita* con esito negativo, determinata dall'abbandono volontario del *giocatore* in qualsiasi fase della *Scalata*.

Tale stato viene impostato quando:

- Il *giocatore* decide di terminare l'intera *scalata* senza averla completata

➤ **FINISHED** — Stato conclusivo per *Completamento*

Rappresenta il completamento con successo dell'intera *scalata*, previsto dal superamento dell'ultimo *livello* della stessa.

Tale stato viene impostato quando:

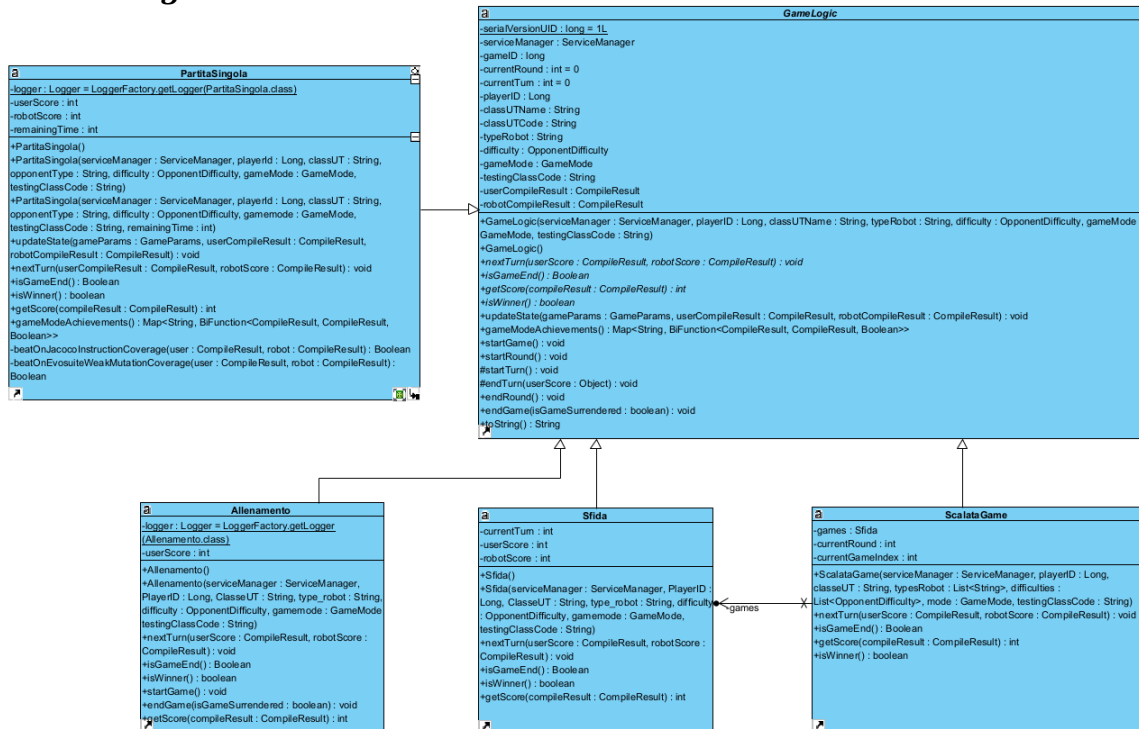
- Il *giocatore* supera l'ultimo *livello* della *scalata* in questione

3.2 Diagramma delle classi (modulo T56)

3.2.1 Diagramma di "GameLogic"

Diagramma originario

Nel modello iniziale tutte le classi effettive di gioco ereditavano direttamente dalla classe astratta **GameLogic**:



Problemi rilevati

La struttura iniziale presentava alcune *criticità strutturali*.

I *livelli* della *modalità Scalata* erano gestiti tramite un array di oggetti **Sfida**. Tuttavia, è emersa una ridondanza superflua tra la classe **Sfida** e la classe **PartitaSingola**.

Sebbene si fosse ipotizzato di sostituire **Sfida** con un insieme di **PartitaSingola**, permaneva una problematica legata all'**ereditarietà**: **ScalataGame**, specializzando **GameLogic**, ne ereditava inevitabilmente gli attributi quali:

- *classeUTName*
- *typeRobot*
- *difficulty*
- ...

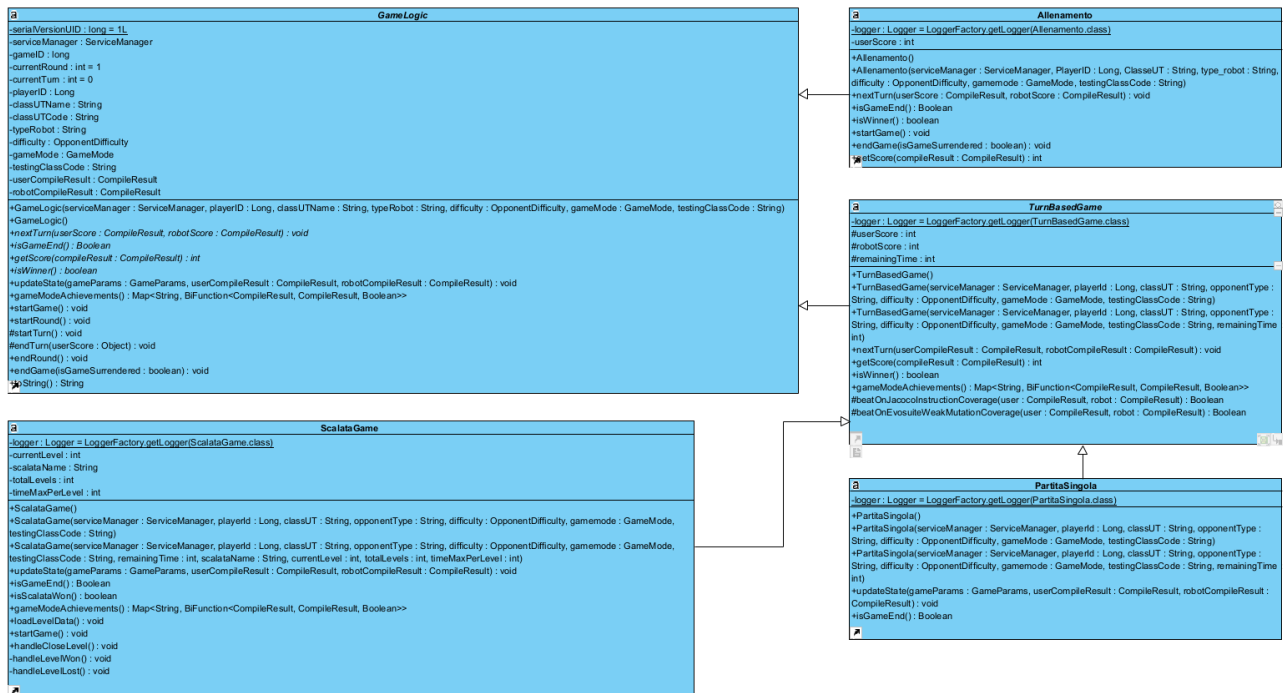
Nel contesto della *Scalata*, tali attributi specifici del *livello* avrebbero dovuto assumere valori di *default* fittizi oppure essere una copia di quelli presenti nel *livello* gestito come **Sfida** o **PartitaSingola**.

È stata altresì scartata la *refattorizzazione* della classe base **GameLogic**: l'alto accoppiamento (in particolare nella creazione della *sessione*) avrebbe comportato un impatto eccessivo sul resto del codice. Si è quindi optato per una soluzione capace di preservare l'integrità di **GameLogic** senza modificarla, garantendo al contempo la corretta gestione degli attributi specifici.

Soluzione adottata

La strategia risolutiva ha previsto la ridefinizione di *ScalataGame* come estensione diretta e autonoma di *GameLogic*, eliminando così le dipendenze pregresse dalle classi Sfida o l'ipotesi di PartitaSingola. Tuttavia, tale approccio avrebbe comportato la duplicazione di metodi e logiche comuni a PartitaSingola.

Per ovviare a questo problema e centralizzare il codice condiviso, è stata introdotta la classe astratta *TurnBasedGame*. Il nuovo diagramma delle classi, riportato di seguito, illustra la struttura finale:



L'architettura del *modulo T56* è stata quindi oggetto di un significativo intervento di *refactoring*, mirato a:

- Eliminare le ridondanze strutturali della versione precedente
- Aumentare la coesione interna delle classi
- Garantire una maggiore estensibilità per future modalità di gioco

➤ *GameLogic* (classe astratta di base)

La classe *GameLogic* resta inalterata mantenendo il ruolo di fondamento astratto per tutte le modalità di gioco.

➤ **TurnBasedGame (nuova classe astratta intermedia)**

La principale innovazione architetturale consiste nell'introduzione della classe astratta *TurnBasedGame*, che estende *GameLogic* e accorpa tutta la logica comune a *PartitaSingola* e *Scalata*.

A questa classe sono state assegnate le seguenti *responsabilità*:

- **Gestione del flusso dei turni:** mediante l'implementazione centralizzata di *nextTurn*
- **Mantenimento dei punteggi:**
 - *userScore*
 - *robotScore*
 - *remainingTime* (tempo residuo)
- **Valutazione degli esiti:** tramite la logica di confronto basata sulle metriche di coverage (*JaCoCo/EvoSuite*)

Questa scelta riduce significativamente la duplicazione del codice e aumenta la manutenibilità complessiva del sistema.

➤ **PartitaSingola**

La classe *PartitaSingola* è stata *rifattorizzata* affinché erediti direttamente dalla nuova classe astratta *TurnBasedGame*.

Tale modifica ha comportato lo spostamento (*pull-up*) di metodi e attributi condivisi verso la *superclasse*, sottraendoli alla gestione diretta della sottoclasse.

Questa specializzazione incrementa la chiarezza semantica dell'architettura e riduce significativamente le responsabilità di *PartitaSingola*, focalizzandola esclusivamente sulle sue specificità.

➤ **ScalataGame**

La classe *ScalataGame* è stata definita come estensione di *TurnBasedGame*, specializzandone il comportamento base per integrare le dinamiche caratteristiche della modalità a *livelli*.

Le principali implementazioni specifiche riguardano:

- **La gestione della progressione:** il monitoraggio dello stato di avanzamento è affidato agli attributi
 - *currentLevel*
 - *totalLevels*
- **Il controllo del flusso di gioco:** la logica di transizione è centralizzata nel metodo *handleCloseLevel*, il quale, coadiuvato dai metodi ausiliari
 - *handleLevelWon*
 - *handleLevelLost*

Orchestra l'evoluzione della *partita* e il passaggio tra i *livelli*

➤ Allenamento

La modalità *Allenamento* è stata mantenuta **isolata** rispetto alla nuova gerarchia basata su *TurnBasedGame*, poiché non prevede un confronto competitivo né una dinamica a *turni* analoga.

Le modifiche principali sono:

- Conferma dell'ereditarietà diretta da *GameLogic*
- Preservazione di una struttura autonoma che evita l'introduzione di comportamenti non pertinenti alla natura non competitiva della modalità

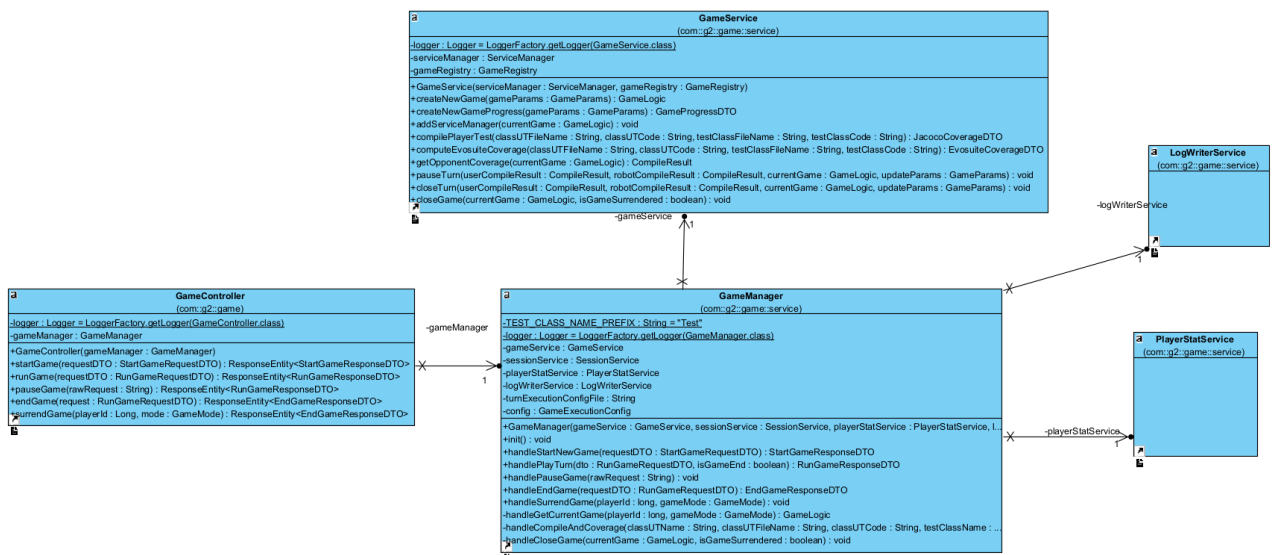
Questo isolamento garantisce che la modalità *Allenamento* rimanga coerente con il suo scopo originario.

3.2.2 Diagramma di “GameManager&Controller”

Diagramma delle classi iniziale

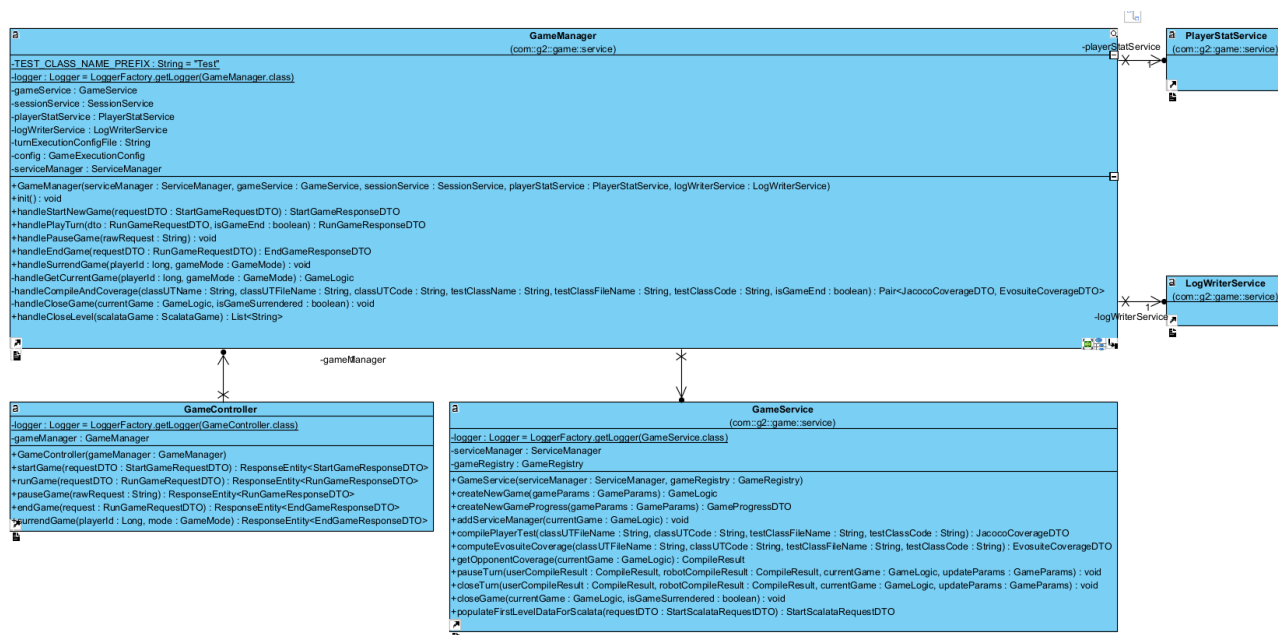
Prima di descrivere le evoluzioni e le modifiche apportate all'architettura del sistema, si riporta di seguito il diagramma delle classi di partenza, che rappresenta la versione iniziale del modello progettuale.

Questo diagramma costituisce il riferimento originario su cui si sono basate le successive scelte architetture e consente di comprendere meglio le motivazioni che hanno portato all'introduzione di nuove *responsabilità* e all'estensione di alcune componenti del sistema.



Soluzione adottata

La modifica del diagramma delle classi è stata guidata dall'introduzione di una gestione più granulare del flusso di gioco, in particolare per supportare la *modalità Scalata*, che richiede una distinzione chiara tra la chiusura di un *singolo livello* e la conclusione dell'intera partita.



Di seguito vengono descritte in modo dettagliato le variazioni apportate ai principali componenti architetturali.

➤ *GameManager (Service Layer)*

L'obiettivo è stato quello di estendere i metodi già esistenti in *GameManager* per gestire anche la *modalità Scalata*, mantenendo la retrocompatibilità con le altre modalità (*Partita Singola*, *Allenamento*). Questa scelta ha permesso di lasciare inalterato il *GameController*, in quanto dal *frontend* i metodi chiamati sono i medesimi, ma la logica interna si adatta automaticamente alla modalità di gioco.

Metodi non modificati

- ***handlePlayTurn()***: la compilazione e il calcolo della copertura del codice sono indipendenti dalla modalità di gioco
- ***handlePauseGame()***: il salvataggio dello stato della *sessione* in *Redis* funziona allo stesso modo per tutte le modalità



Modifiche introdotte

handleStartNewGame()

Il metodo è stato esteso per gestire il caricamento dinamico dei dati del primo *livello* nella *modalità Scalata*.

A differenza delle altre modalità, dove il *frontend* invia tutti i parametri necessari:

- *Classe da testare*
- *Tipo di robot*
- *Difficoltà*
- *Tempo*

Nella *Scalata* i dati del primo livello devono essere recuperati dal modulo T1.

handleEndGame()

La principale differenza tra la *modalità Scalata* e quelle già esistenti emerge alla fine di ogni *round*: mentre nella modalità *Partita Singola* il gioco termina sempre, nella *Scalata* può esserci una progressione.

Il metodo in questione è stato esteso per:

- **Distinguere la modalità di gioco:** tramite controllo sul *tipo* di istanza, il comportamento si adatta automaticamente
- **Gestire la progressione:** nel caso della *modalità Scalata*, il metodo in questione delega al metodo ausiliario *handleCloseLevel()* di prendere una delle seguenti decisioni
 - Avanzare al *livello* successivo
 - Riprovare il *livello* raggiunto dal *giocatore*
 - Terminare l'intera *scalata*
- **Mantenere il comportamento originario:** per le altre modalità il comportamento di chiusura del gioco resta invariato

Questa architettura evita la duplicazione di codice e mantiene il flusso principale leggibile.

handleCloseLevel() - Nuovo metodo

Questo nuovo metodo ausiliario incapsula tutta la logica di transizione tra i *livelli* della *Scalata*

- **In caso di completamento del round:** delega al metodo *handleLevelWon* di *ScalataGame*
 - L'assegnazione delle ricompense (*punti esperienza e achievement*)
 - Il caricamento dei dati del *livello* successivo
 - L'aggiornamento dello *stato* della *partita*
 - L'aggiornamento della *sessione* con i dati del nuovo *livello*
- **In caso di fallimento del round:** delega al metodo *handleLevelLost* di *ScalataGame*
 - Il ripristino del *tempo*
 - La registrazione del *tentativo* appena fallito
 - Il salvataggio della *sessione* di gioco per permettere un nuovo *tentativo*



➤ *GameService (Business Logic Layer)*

La classe *GameService* è stata modificata per supportare l'inizializzazione della *modalità Scalata*. È stato aggiunto il metodo:

populateFirstLevelDataForScalata(requestDTO: StartScalataRequestDTO): StartScalataRequestDTO

Motivazione della modifica

Con l'introduzione della *modalità Scalata*, si è resa necessaria una *fase di preparazione dei dati iniziali* più articolata rispetto a quella delle modalità di gioco *tradizionali*.

Il nuovo metodo consente al *service* di:

- Pre-caricare e arricchire i dati relativi al *primo livello*
- Recuperare informazioni da servizi esterni o componenti di supporto
- Garantire che il *gioco* venga avviato con uno stato iniziale coerente e completo

Questa scelta progettuale rafforza il ruolo del *GameService* come *strato responsabile* della *logica di business*, evitando che tali responsabilità ricadano su livelli architetturali non appropriati.

➤ *GameController (API Layer)*

La classe *GameController* non presenta modifiche “*visibili*” nelle firme dei metodi esposti nel diagramma delle classi. I metodi attualmente disponibili rimangono invariati:

- *startGame*
- *runGame*
- *pauseGame*
- *endGame*
- *surrendGame*

Considerazioni progettuali

L'assenza di modifiche nelle interfacce pubbliche del *controller* indica una scelta consapevole di *mantenere stabile le API verso il frontend*, nonostante le evoluzioni interne del sistema. Eventuali modifiche di comportamento sono state confinate ai livelli sottostanti (*Service e Manager*), preservando così:

- Retrocompatibilità
- Separazione delle responsabilità
- Semplicità dell'interazione lato *client*

Conclusione

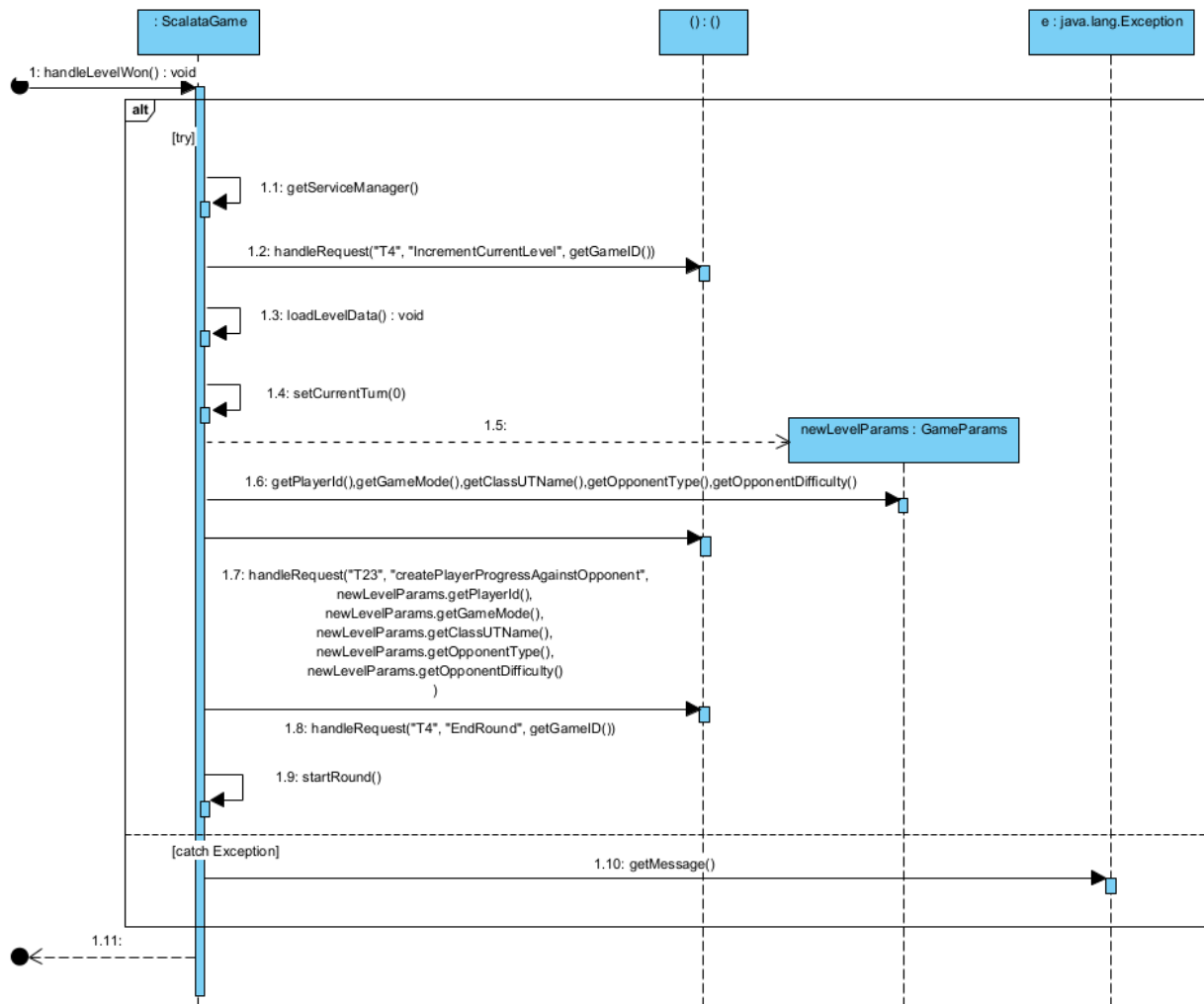
In sintesi, l'architettura è stata evoluta per:

- Introdurre il concetto di **transizione di livello** attraverso il metodo *handleCloseLevel*
- Supportare un'**inizializzazione specializzata** della *modalità Scalata* tramite *populateFirstLevelDataForScalata*
- Mantenere invariata l'interfaccia del **GameController** verso il *frontend*

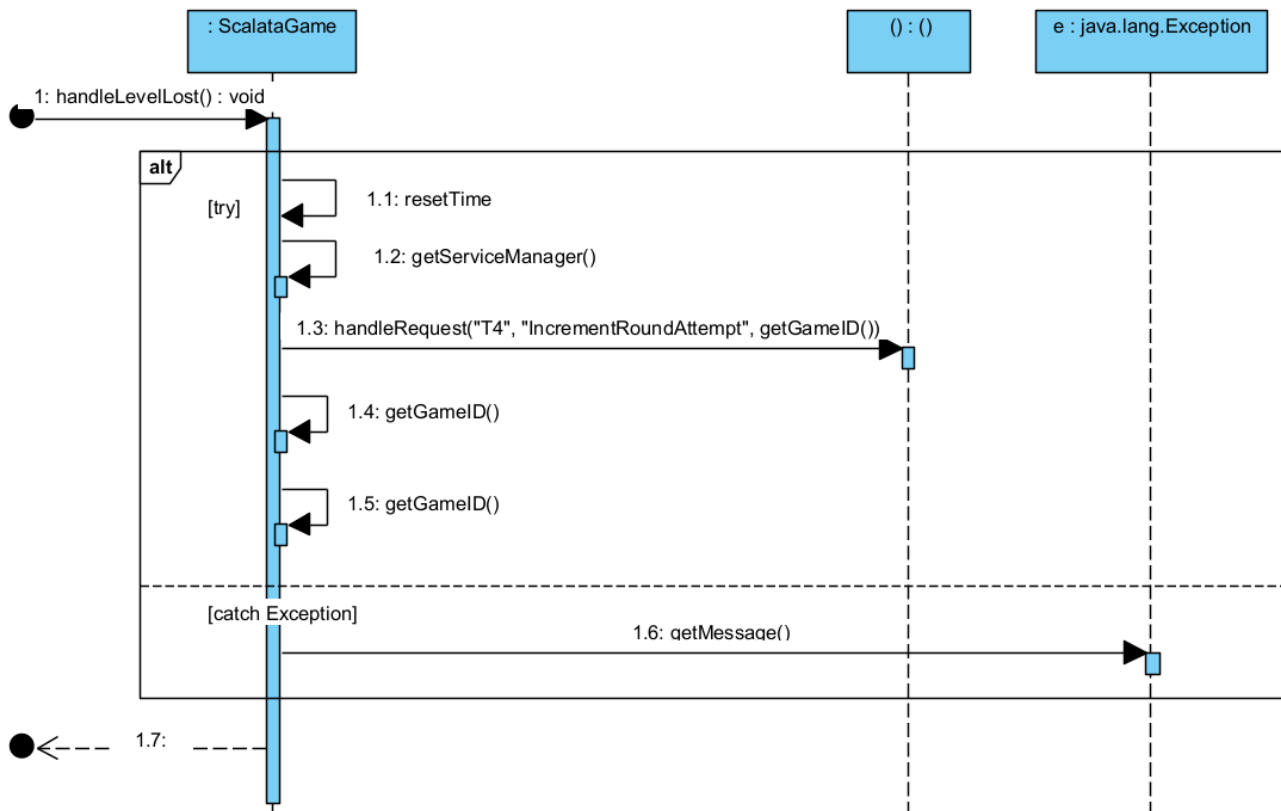
Queste modifiche migliorano la flessibilità, la manutenibilità e la chiarezza del modello, rendendo il sistema più adatto a gestire modalità di gioco complesse e future estensioni.

3.3 Diagramma di Sequenza

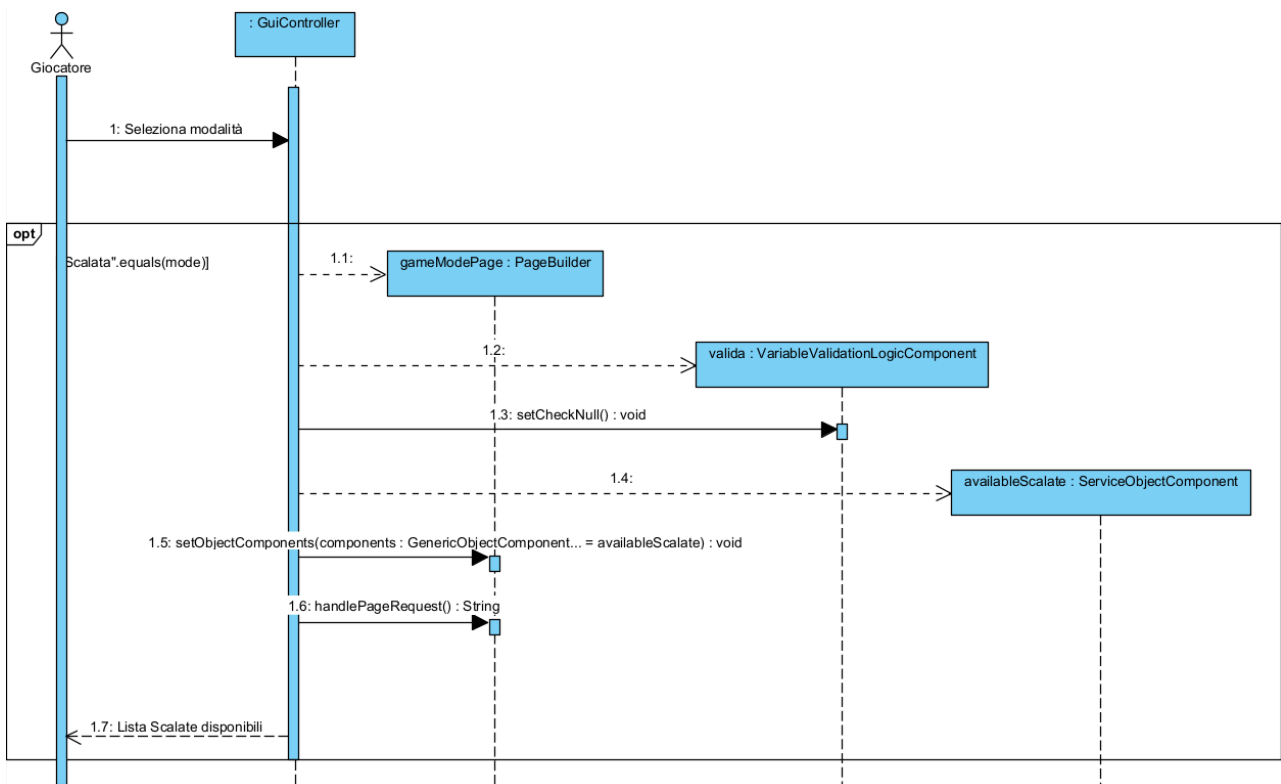
3.3.1 Diagramma di "HandleLevelWon"



3.3.2 Diagramma di “HandleLevelLost”



3.3.3 Diagramma di “SelezioneScalata”



3.4 Gestione delle Interfacce: *Estensioni e Modifiche*

3.4.1 Lista Scalate

Descrizione Funzionale

Questa interfaccia permette di recuperare l'elenco completo delle "Scalate".

Viene invocata dal **microservizio T56** quando un utente accede alla sezione *giochi*, per verificare quali *scalate* sono disponibili da mostrare nel *frontend*.

Dettagli Tecnici

- **Provider:** T1 (*AdminService*)
- **Consumer:** T56
- **Metodo HTTP:** GET
- **Endpoint (Interno):** /scalata/getAll
- **Endpoint (Gateway):** /adminService/scalata/getAll

Request

- **Parametri:** nessuno (*La richiesta non necessita di body o query params*)

Errori

- Nessuno (*ritorna [] se non ci sono scalate*)

Esempio di Utilizzo (*in memoria è presente una scalata con tre livelli*)

- **Richiesta:** GET /adminService/scalata/getAll

```
Esempio di risposta(Successo - 200 OK)

1 [
2 {
3   "scalataName" : "Ascesa" ,
4   "nome utente" : "Antonio" ,
5   "descrizione" : "Primo esempio di Scalata" ,
6   "numeroLivelli" : 3 ,
7   "elencoLivelli" : [
8     { "avversario" : "ObjectId(693d7e1c26f1a11f12b355d6)" , "tempoMax" : 60 } ,
9     { "avversario" : "ObjectId(693d7e7026f1a11f12b355ec)" , "tempoMax" : 90 } ,
10    { "avversario" : "ObjectId(693d7e3726f1a11f12b355e6)" , "tempoMax" : 120 }
11  ]
12 }
13 ]
```

3.4.2 Recupero Dettagli Livello Scalata

Descrizione Funzionale

Questa interfaccia viene invocata dal **microservizio T56** per ottenere i dettagli specifici del *livello* corrente all'interno di una *scalata*

Dettagli Tecnici

- **Provider:** T1 (*AdminService*)
- **Consumer:** T56
- **Metodo HTTP:** GET
- **Endpoint (Interno):** /scalata/getLevel/{scalataName}/{number}
- **Endpoint (Gateway):** /adminService/scalata/getLevel/{scalataName}/{number}

Request

- **Path Parameters:**

Parametro	Tipo	Descrizione
<i>scalataName</i>	String	Nome univoco della scalata
<i>number</i>	Int	Numero progressivo necessario per tracciare il livello da recuperare

Nota: *number* corrisponde al *livello* corrente a cui è arrivato il *giocatore decrementato* (*currentLevel-1*) per accedere alla posizione corretta del *livello* nella lista.

Errori

- **404 Not Found:** *scalata non trovata*
- **400 Bad Request:** *livello non valido (fuori range)*
- **500 Internal Server Error:** *dati corrotti o errore database*

Esempio di Utilizzo

- GET /adminService/scalata/getLevel/Ascesa/0

```
Esempio di risposta(Successo - 200 OK)

1 //JSON
2 {
3   "tempoMax": 60,
4   "opponent": {
5     "_id": "ObjectId(693d7e1c26f1a11f12b355d6)",
6     "classUT": "FTPFile",
7     "opponentDifficulty": "EASY",
8     "opponentType": "EvoSuite"
9   }
10 }
```

3.4.3 Incremento del *currentlevel* di una partita

Descrizione Funzionale

Questa interfaccia permette di aggiornare lo stato di una *partita* in *modalità "Scalata"*, incrementando di un'unità il valore del *livello* corrente (*current_level*).

Viene invocata dal *microservizio T56* al termine di ogni *livello* superato con successo dal *giocatore*.

Dettagli Tecnici

- **Provider:** *T4 (GameRepo)*
- **Consumer:** *T56*
- **Metodo HTTP:** *PUT*
- **Endpoint (interno):** */games/{gameId}/rounds/last/increment-attempt*
- **Endpoint (Gateway):** */gamerepo/games/{gameId}/current-level-increment*

Request

- **Parametri:** *gameId (Long)* -> ID univoco della partita

Codici di Stato

- **200 OK:** *aggiornamento effettuato con successo. Restituisce la partita aggiornata*
- **404 Not Found:** *nessuna partita trovata con il gameId specificato*

Esempio di Utilizzo

- **Richiesta:** *si richiede l'incremento del livello per la partita con ID 12345*
 - *HTTP PUT /gamerepo/games/12345/current-level-increment*

```
1 //JSON
2 {
3   "id": 12345,
4   "gameMode": "SCALATA",
5   "currentLevel": 3,           // Valore aggiornato
6   "scalataName": "Scalata Facile",
7   "status": "IN_PROGRESS",
8   "players": [42],
9   "createdAt": "2025-12-12T10:30:00",
10  "closedAt": null
11 }
```

Nota

Risposta (200 OK): *Restituisce l'oggetto Game aggiornato (notare currentLevel aggiornato).*

3.4.4 Incremento Tentativo Round [Flusso: T56 (GameEngine) -> T4 (GameRepo)]

Descrizione Funzionale

Questa interfaccia incrementa il contatore dei *tentativi* (*roundNumber*) associato al *round* corrente (l'ultimo *round* non ancora chiuso).

Viene invocata dal **microservizio T56** quando un *giocatore* fallisce un *livello* in *modalità "Scalata"* e decide di riprovare, permettendo al sistema di tracciare quanti *tentativi* sono stati effettuati sullo stesso *livello*.

Dettagli Tecnici

- **Provider:** *Provider: T4 (GameRepo)*
- **Consumer:** *T56 (GameEngine)*
- **Metodo HTTP:** *PUT*
- **Endpoint (Interno):** */games/{gameId}/rounds/last/increment-attempt*
- **Endpoint (Gateway):** */gamerepo/games/{gameId}/rounds/last/increment-attempt*

Request

- **Parametri:** *gameId (Long) -> ID univoco della partita*

Codici di Stato

- **200 OK:** *aggiornamento riuscito. Restituisce il round aggiornato*
- **400 Bad Request:** *la partita non è nello stato IN_PROGRESS (non è possibile modificare una partita chiusa)*
- **404 Not Found:** *partita non trovata oppure non esiste nessun round attivo (aperto) per questa partita*

Esempio di Utilizzo

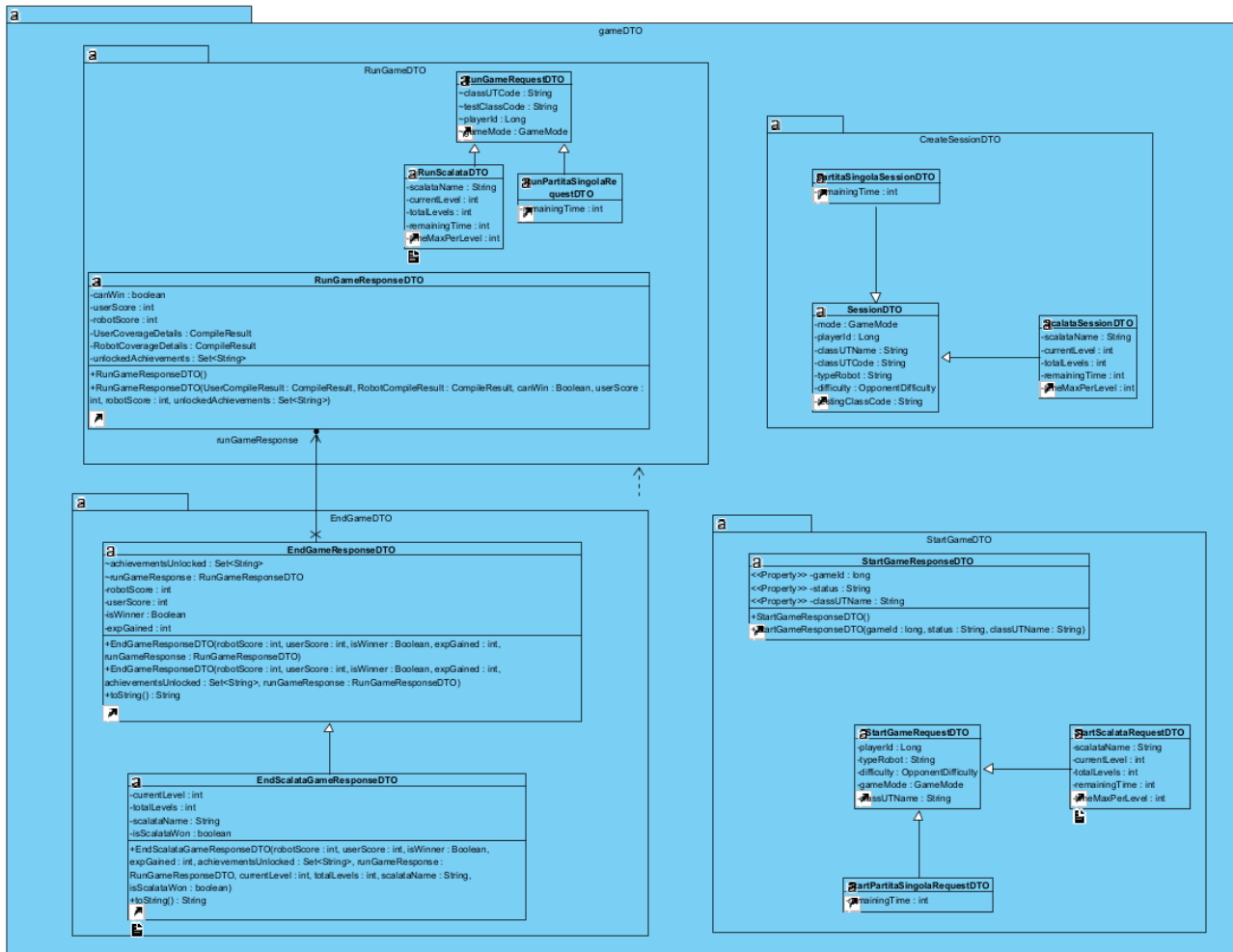
- **Richiesta:** *si richiede l'incremento del tentativo per il round attivo della partita ID 12345*
 - *HTTP PUT /gamerepo/games/12345/rounds/last/increment-attempt*

```
1 //JSON
2 {
3   "id": 789,
4   "roundNumber": 2,           // Valore incrementato (era 1)
5   "classUT": "FTPFile",
6   "type": "EvoSuite",
7   "difficulty": "EASY",
8   "createdAt": "2025-12-12T10:35:00",
9   "closedAt": null
10 }
```

Nota: *risposta (200 OK)* restituisce l'oggetto *Round* aggiornato (con il numero di tentativi incrementato).

4. Implementazione

4.1 Package “game/gameDTO”



L'evoluzione della logica applicativa introdotta dalla modalità *Scalata* ha richiesto un'estensione mirata del *Data Transfer Layer*, mantenendo al contempo la piena compatibilità con le modalità preesistenti.

A tal fine, si è adottata una strategia basata sull'ereditarietà, che consente di specializzare i *DTO* coinvolti nel ciclo di vita della partita senza modificare o interrompere i contratti già in uso per la *Partita Singola*.

Il nuovo schema *DTO* preserva la stabilità delle *API* esistenti, introducendo classi derivate che arricchiscono il *payload* con i dati aggiuntivi necessari per la gestione della progressione su più livelli.

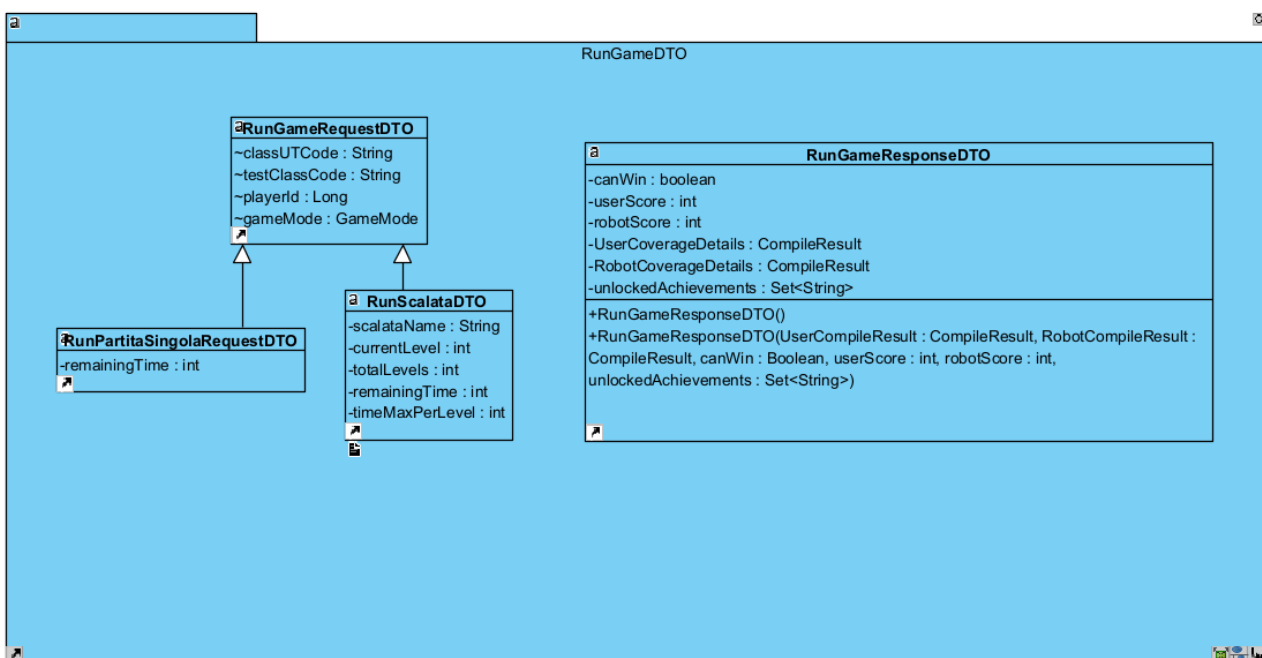
Il diagramma rappresenta il package **gameDTO**, che funge da contenitore principale per tutti i sottopackage relativi alla gestione dei dati della partita.

All'interno di *gameDTO* sono definiti i package specifici per le principali operazioni di gioco, tra cui:

- ***RunGameDTO***: gestisce i dati relativi all'esecuzione della *partita*
- ***RunGameResponseDTO***: contiene i dati di risposta generati al termine di un *turno* o di un'azione di gioco
- ***CreateSessionDTO***: contiene le informazioni necessarie per l'inizializzazione di una nuova sessione
- ***EndGameDTO***: raccoglie i dati finali della *partita*, inclusi risultati e punteggi
- ***StartGameDTO***: organizza i dati richiesti all'avvio della *partita*

Il diagramma evidenzia le relazioni tra i sottopackage, mostrando come i diversi dati vengono strutturati e correlati all'interno del contesto di *gameDTO*.

4.1.1 Package “*RunGameDTO*”



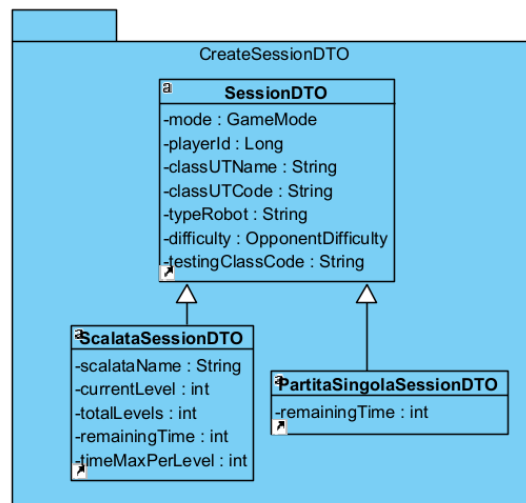
Per gestire l'esecuzione di ogni turno della scalata (*compilazione ed esecuzione dei test*), è stata introdotta la classe ***RunScalataDTO***, estendendo *RunGameRequestDTO*.

Questa specializzazione trasmette al backend il contesto completo dello stato della scalata, includendo:

- *currentLevel*
- *totalLevels*

Tali informazioni risultano fondamentali affinché il *Game Engine* possa validare correttamente le azioni del *giocatore* in relazione al *livello* corrente ed eseguire la logica di avanzamento in modo coerente.

4.1.2 Package “CreateSessionDTO”



Per rappresentare correttamente lo stato iniziale di una *sessione Scalata*, è stata introdotta la classe **ScalataSessionDTO**, derivata da *SessionDTO*.

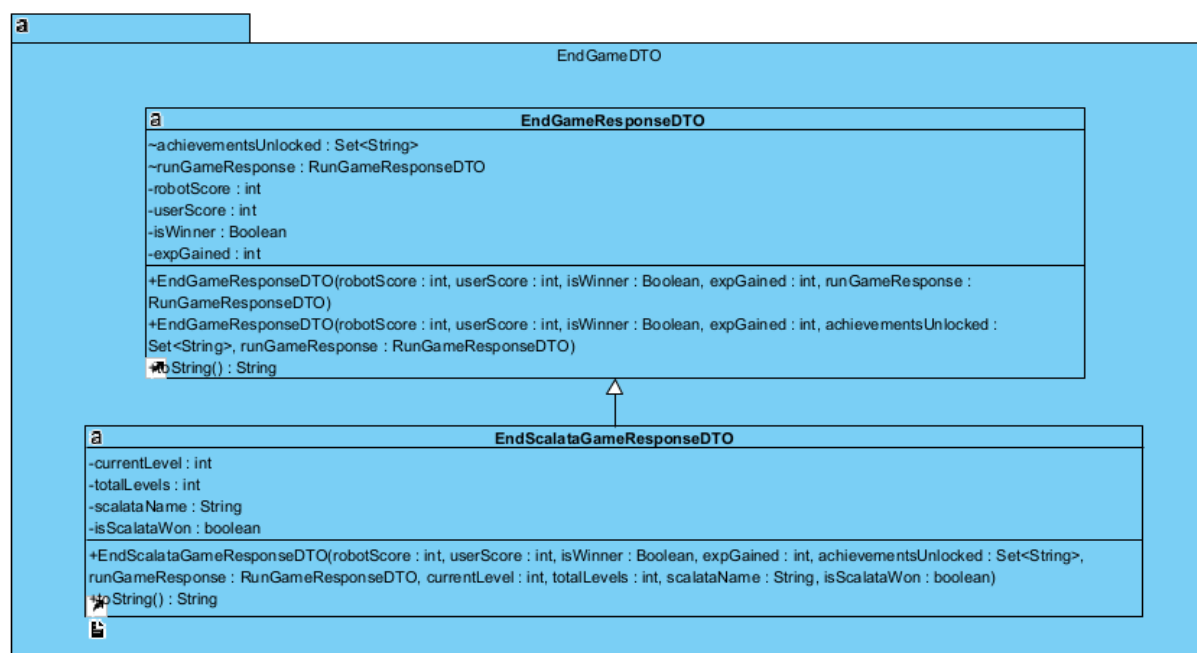
Questa specializzazione consente di trasportare informazioni aggiuntive non richieste dalla *modalità singola*.

Differenze principali:

- La *PartitaSingolaSessionDTO* si limita alla gestione del tempo rimanente
- **ScalataSessionDTO** include dati strutturali essenziali per la progressione:
 - *currentLevel*
 - *totalLevels*
 - *timeMaxPerLevel*
 - *scalataName*

Tale estensione abilita il *frontend* a visualizzare correttamente lo stato della scalata sin dalla creazione della sessione.

4.1.3 Package “EndGameDTO”



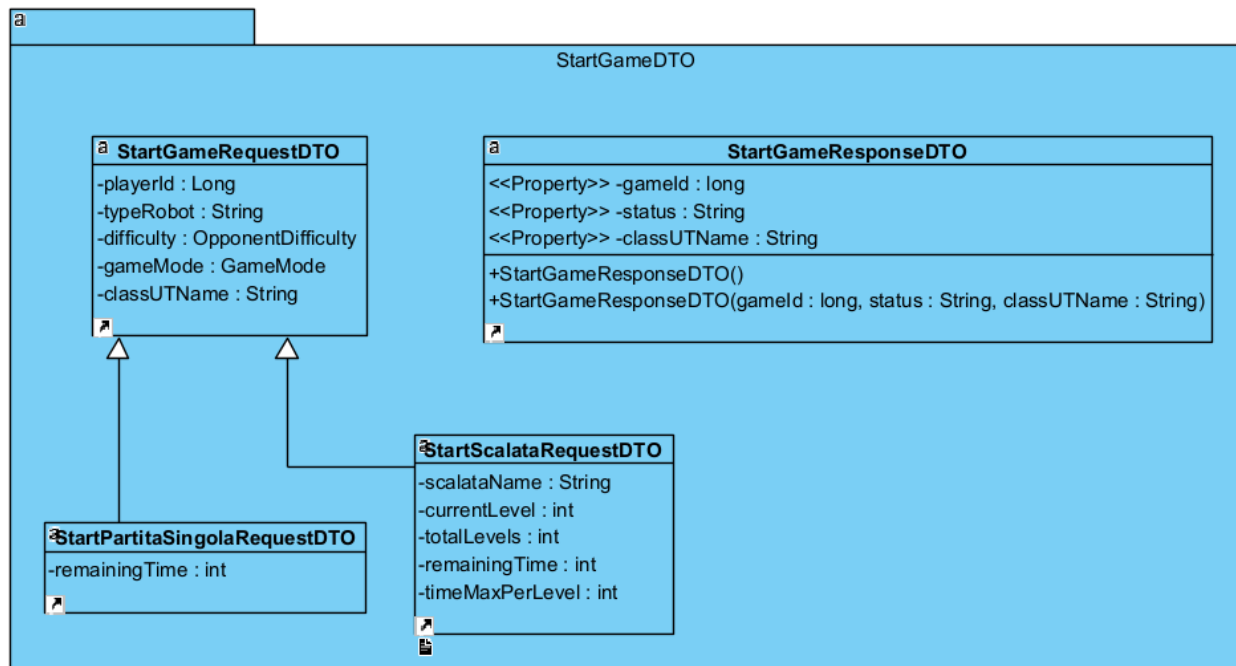
La conclusione della partita è stata adattata attraverso l'introduzione di **EndScalataGameResponseDTO**, che estende il *DTO* standard di terminazione.

Oltre alle informazioni già previste per le *modalità tradizionali*, questa classe:

- Fornisce il flag booleano **isScalataWon**: che specifica se il giocatore ha completato con successo l'intera sequenza di *livelli*
- Restituisce un riepilogo del *livello* raggiunto rispetto al totale

Questi dati permettono al sistema di sintetizzare l'esito complessivo della *scalata*, distinguendo tra completamento totale e interruzione anticipata.

4.1.4 Package “StartGameDTO”



La richiesta di avvio della *partita* è stata specializzata mediante la creazione di **StartScalataRequestDTO**, che estende **StartGameDTO**.

Questa scelta consente al client di trasmettere parametri specifici per l’inizializzazione della *scalata* senza interferire con il funzionamento di **StartPartitaSingolaRequestDTO**.

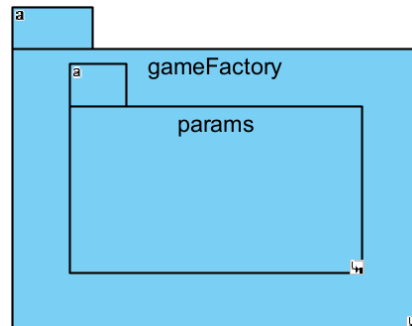
Tra i dati introdotti figurano:

- Configurazione del livello iniziale
- Selezione della scalata
- Parametri necessari alla costruzione della progressione

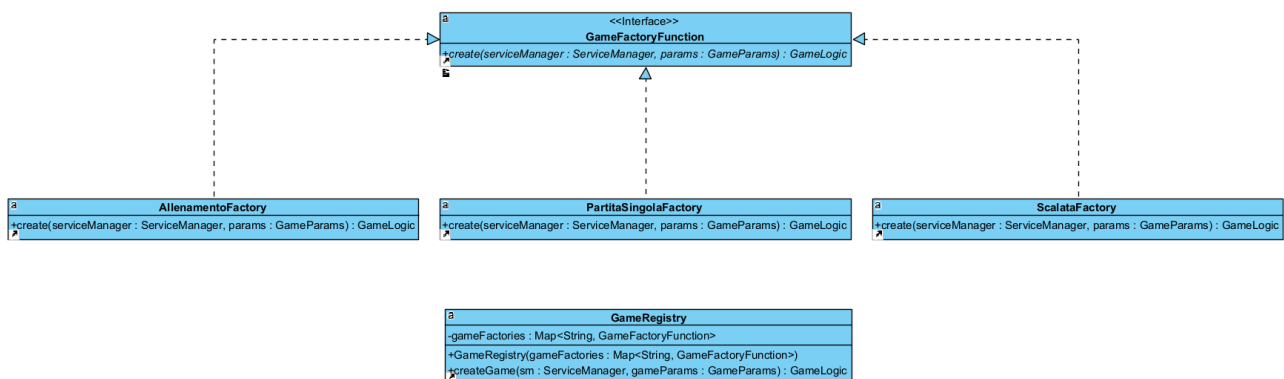
La distinzione tra i due *DTO* rende il contratto più esplicito e migliora la chiarezza semantica delle *API*.

4.2 Package “game/gameFactory”

Nel processo di integrazione della modalità *Scalata*, l’architettura esistente è stata preservata e al contempo estesa in modo controllato, garantendo piena compatibilità con le modalità già presenti. Le principali modifiche hanno riguardato diversi aspetti.



4.2.1 Package “gameFactory”

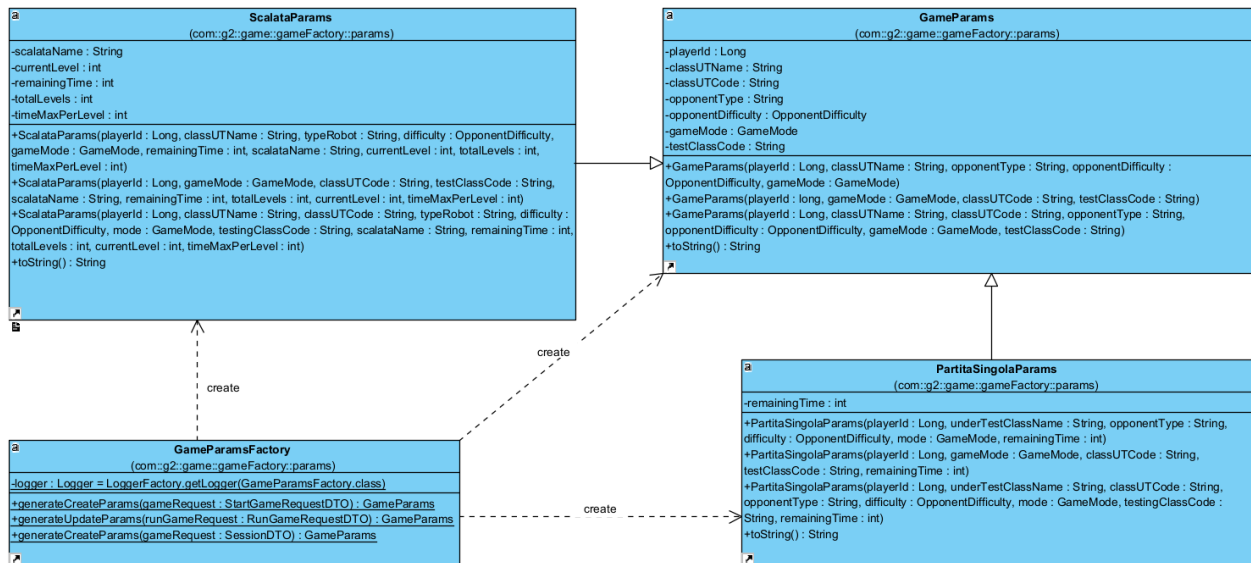


Factory Pattern

È stata introdotta la classe **ScalataFactory**, che implementa l’interfaccia comune *GameFactoryFunction*.

A differenza della versione originaria, limitata alle factory per *Allenamento* e *Partita Singola*, questa aggiunta consente al *GameRegistry* di istanziare in modo trasparente e uniforme la logica specifica della *modalità Scalata*, mantenendo coerenza e modularità nel processo di creazione delle partite.

4.2.2 Package “params”



Parameter Object

Per gestire i dati di inizializzazione propri della *Scalata*, è stata creata la classe ***ScalataParams***, derivata da *GameParams*.

Mentre la struttura preesistente gestiva esclusivamente parametri generici o relativi alla *Partita Singola*; *ScalataParams* incapsula informazioni fondamentali come

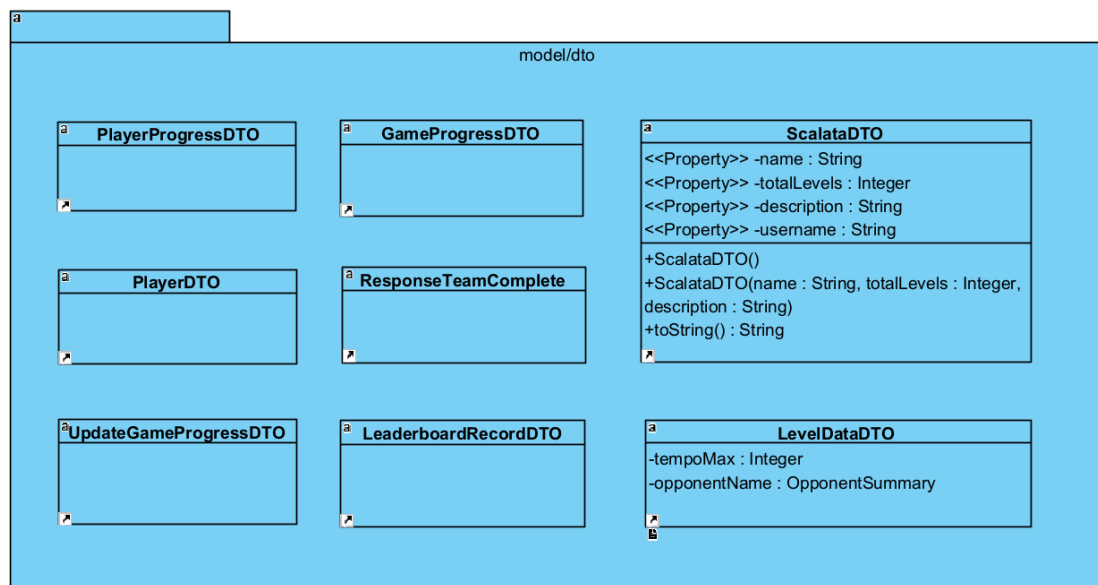
- *currentLevel*
- *totalLevels*
- *timeMaxPerLevel*

La *GameParamsFactory* è stata aggiornata per generare correttamente queste istanze a partire dai *DTO* di input, assicurando un’inizializzazione coerente della *modalità Scalata*.

4.3 Package “model/dto”

Sono stati introdotti due nuovi *DTO* fondamentali per definire l'oggetto “*Scalata*” proveniente dal servizio di *backend*

- **ScalataDTO:** rappresenta l'intera *sfida*, contenente metadati globali
 - *name*
 - *description*
 - *totalLevels*
 - *username*
- **LevelDataDTO:** definisce il singolo *step* della *scalata*, incapsulando le informazioni specifiche per quel *livello*
 - *className*
 - *tempoMax*
 - *opponentName*



5. Sviluppi Futuri

Attualmente, la *modalità Scalata* funziona correttamente solo se gestita tramite sessione *Redis*.

Descrizione del problema

Se il *giocatore* dovesse chiudere il *browser*, si potrebbero presentare due alternative:

- La sessione *Redis* scade per *timeout*
- Il *giocatore* riaccede alla *partita* lasciata in sospeso

Il sistema, attualmente, non è in grado di recuperare lo stato della *partita* e la *scalata* non può essere ripresa correttamente.

Soluzione proposta (non implementata)

Per gestire il recupero dello stato della *scalata* in assenza di sessione *Redis attiva*, è sufficiente interrogare il *modulo T4* e ricostruire lo stato di gioco attraverso il seguente *flusso*:

1. Interrogazione al *modulo T4*

Recupero dell'ultima partita *Scalata* del giocatore tramite:

- *GET /games/player/{playerId}* (filtrando per tipo *Scalata*)

2. Verifica dello stato del gioco

- Se lo stato è *FINISHED* o *SURRENDERED* non esiste una *partita* attiva da recuperare
- Se lo stato è *STARTED* o *IN_PROGRESS* esiste una *partita* attiva che può essere recuperata

3. Recupero dei parametri della *scalata*

Dall'entità *Game* recuperata dal *modulo T4*, estrarre:

- *gameName*: il nome della *scalata* in corso
- *currentLevel*: il livello corrente raggiunto dal giocatore

4. Interrogazione al *modulo T1*

Utilizzare i dati estratti per interrogare il *modulo T1* tramite:

- *GET /scalata/{gameName}/level/{currentLevel}*

Ottenendo così i metadati del *livello* corrente, ossia:

- *Classe Under Test*
- *Robot*
- *Difficoltà*
- *Tempo massimo*

5. Ricostruzione dello stato di gioco

Istanziare un nuovo oggetto *ScalataGame* con

- I dati recuperati dal *modulo T4*:
 - *ID partita*
 - *Stato*
 - *CurrentLevel*
- I parametri del livello recuperati dal *modulo T1*
- Lo stato di sessione salvato nuovamente in *Redis*

Conclusione

Questa soluzione permette di ripristinare completamente lo stato della *scalata* senza richiedere modifiche significative all'architettura esistente, sfruttando le informazioni già persistite nel *modulo T4* e i metadati disponibili nel *modulo T1*.

Un possibile sviluppo futuro consiste nell'impiego dell'attributo ***gameName*** presente nel *modulo T4* — attualmente valorizzato come null nel caso di *partite singole* — al fine di consentire al *giocatore* di assegnare un nome personalizzato alla partita giocata.

6. Testing

6.1 Test funzionale

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese
1	Vittoria di un generico livello intermedio	• Tempo disponibile • Criteri soddisfatti • Livello intermedio	• Il giocatore è in una partita attiva affrontando un generico livello diverso dall'ultimo • Timer > 0 • Codice di test del giocatore è corretto con coverage maggiore/uguale di quella del Robot	Invio della classe di test	Visualizzazione pop-up: <i>"Livello Superato"</i>	Livello corrente incrementato
2	Vittoria finale della Scalata	• Tempo disponibile • Criteri soddisfatti • Ultimo livello	• Il giocatore è in una partita attiva affrontando l'ultimo livello • Timer > 0 • Codice di test del giocatore è corretto con coverage maggiore/uguale di quella del Robot	Invio della classe di test	Visualizzazione pop up: <i>"Scalata Completata"</i>	• Scalata memorizzata come terminata • Specifico Achievement sbloccato

Test Case ID	Descrizione	Classi di equivalenza coperte	Pre-condizioni	Input	Output Attesi	Post-condizioni Attese
3	Sconfitta del round per coverage insufficiente	- Tempo disponibile - Criteri di coverage non soddisfatti	- Il giocatore è in una partita attiva affrontando un qualsiasi livello - Timer > 0 - Codice di test del giocatore è corretto con coverage minore di quella del Robot	Invio della classe di test	Visualizzazione pop up: <i>"Obiettivo non raggiunto. Riprova"</i>	Incremento del numero di Round sostenuti dal giocatore per quel determinato livello
4	Sconfitta del round per errore di compilazione	- Tempo disponibile - Errore di Compilazione	- Il giocatore è in una partita attiva affrontando un qualsiasi livello - Timer > 0 - Codice di test del giocatore non compilabile	Invio della classe di test	Visualizzazione pop up: <i>"Obiettivo non raggiunto. Riprova"</i>	Incremento del numero di Round sostenuti dal giocatore per quel determinato livello